

GrupoGame

Do código à explicação

Arquitetura, telas, componentes e decisões.

Teoria aplicada, exemplos comentados e código completo.

Um material para entender, demonstrar e defender o projeto.

01 **ENTENDER**
Pastas, tecnologias e responsabilidades

02 **ACOMPANHAR**
Estado, eventos, navegação e interface

03 **EXPLICAR**
Roteiro, perguntas e código de consulta

Sumário

Use os capítulos para estudar e os marcadores do PDF para localizar cada arquivo do apêndice. A numeração abaixo é a página física do documento.

01. Como estudar este projeto	4
02. Problema, requisitos e escopo	5
03. Referências visuais e leitura das telas	6
04. Tecnologias e o papel de cada uma	9
05. Arquitetura: visão geral	10
06. Pastas, arquivos e decisões de organização	11
07. Configuração e execução	13
08. React, componentes e JSX	14
09. TypeScript e sintaxe que aparece no código	16
10. Props, callbacks, estado e dados derivados	18
11. Dados simulados e relações entre IDs	20
12. Navegação: rotas, pilha e parâmetros	21
13. Inicialização, fontes e tela de Login	23
14. Home: listas, filtro e composição	24
15. Componentes reutilizáveis e suas props	25
16. Detalhes do servidor	26
17. Agendar: uma tela, vários estados	27
18. TextInput e formulário controlado	28
19. Teclado e rolagem: o exemplo.js aplicado	29
20. Seleção de servidor: tela cinco	31
21. Confirmação de saída	32

22. StyleSheet, Flexbox e adaptação de layout	33
23. Acessibilidade, assets e cuidados de interface	35
24. Decisões técnicas e alternativas	36
25. Verificações, limites e demonstração manual	38
26. Roteiro para explicar o projeto ao professor	40
27. Perguntas que podem aparecer na banca	42
28. Exercícios e respostas esperadas	44
29. Glossário e mapa de consulta	46
30. Referências e critérios de evidência	48
A01. Inicialização, fontes e Stack	49
A02. Tela de Login	50
A03. Tela Home	53
A04. Detalhes do servidor	57
A05. Formulário Agendar	60
A06. Cartão de categoria	65
A07. Cartão de partida	67
A08. Cabeçalho	70
A09. Jogador	71
A10. Seleção de servidor	72
A11. Confirmação de saída	74
A12. Dados de Home e categorias	76
A13. Servidores e jogadores	78
A14. Dependências e comandos	80
A15. Configuração do TypeScript	81
A16. Configuração do Expo	82

01. Como estudar este projeto

Este material explica o GrupoGame como um projeto acadêmico de aplicativo: problema, objetivos, arquitetura, dados, telas, decisões de implementação e evidências de verificação. A base é o código local da edição de 24/09/2026, já comentado em português. O nome GamePlay aparece em algumas referências visuais; o projeto local se chama grupogame-app.

O objetivo do aplicativo é demonstrar uma interface para organizar partidas entre grupos de jogadores. O usuário entra pela tela de Login, consulta partidas, vê os detalhes de um servidor e preenche um formulário para agendar. A categoria e o servidor escolhidos são representados por estado local. A interface responde aos toques, mas não existe integração real com Discord nem banco de dados.

Como o material está organizado

1. Entenda o produto e o mapa de pastas antes de decorar linhas de código.
2. Estude componentes, JSX, TypeScript, props e estado. São a base das quatro telas.
3. Acompanhe cada fluxo do toque até a mudança visual.
4. Use o roteiro de apresentação e responda às perguntas sem consultar o texto.
5. Consulte o apêndice para localizar a implementação completa de cada arquivo.

Os exemplos dos capítulos são recortes ou adaptações didáticas identificadas no texto. Eles explicam um conceito isolado e podem omitir imports ou estilos. O apêndice transcreve os 13 arquivos principais e três configurações, incluindo comentários, sem trocar a implementação por exemplos. A numeração lateral do apêndice serve para consulta e não faz parte do código.

A resposta que importa na apresentação

Para cada parte, responda: **o que faz, por que foi feita dessa maneira, de onde vêm os dados e o que acontece depois do toque**. Dizer apenas “isso é um useState” identifica a ferramenta, mas não explica sua responsabilidade.

Exemplo de explicação: “Guardo somente o ID da categoria porque o formulário permite uma única seleção. Cada cartão compara seu ID com esse estado. Quando o usuário toca, o callback atualiza o ID e o React aplica o destaque ao cartão correspondente.”

Não apresente a imagem de referência como se fosse uma captura do aplicativo funcionando. As imagens desta apostila são identificadas como protótipos. Também não confunda uma exportação bem-sucedida com testes de teclado e toque em um celular.

02. Problema, requisitos e escopo

Problema representado

Grupos de jogadores precisam combinar categoria de partida, servidor, data, horário e descrição. O projeto representa essa organização em uma interface móvel. Seu propósito acadêmico é demonstrar composição de componentes, navegação e mudanças de estado, mantendo o código simples de ler e explicar.

Parte	O que foi implementado	Limite da demonstração
Login	Ilustração, apresentação e botão Discord	O botão navega; não autentica
Home	Saudação, categorias, filtro e partidas	Dados locais, sem agenda remota
Detalhes	Servidor, banner, descrição e jogadores	Status de jogadores fictícios
Agendar	Categoria, grupo, campos e teclado	Nada é gravado ou enviado
Lista de servidores	Modal, rolagem, escolha e troca	Seis grupos simulados
Saída	Confirmação e retorno ao Login	Não há sessão real para encerrar

Requisitos técnicos

- Preservar React Native, Expo e TypeScript existentes.
- Usar StyleSheet e Flexbox para construir a interface.
- Aproveitar componentes e bibliotecas já instalados.
- Usar Expo Router para a navegação entre telas.
- Evitar estado global, backend ou abstrações sem necessidade.
- Documentar decisões e manter diferenças visuais conhecidas registradas.

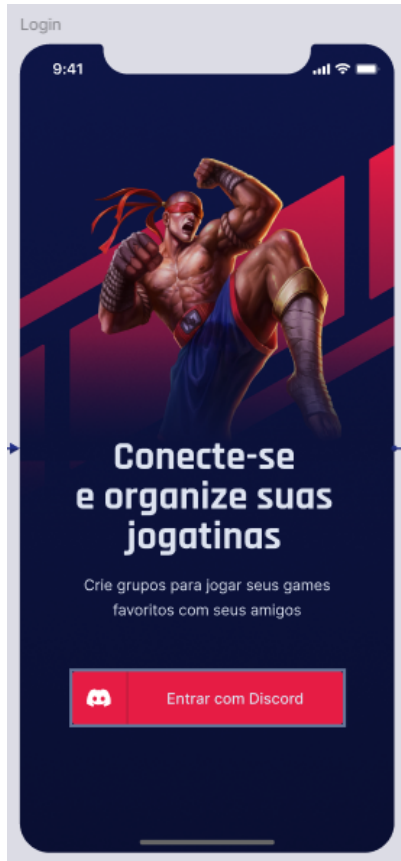
O modal de servidores e a confirmação de saída foram adicionados por solicitações posteriores ao escopo inicial. O estado atual começa com “Selecione um servidor”; Valorosos não é mais uma escolha fixa. As categorias de Agendar começam apagadas e somente a escolhida fica com o destaque normal.

Critério de conclusão acadêmica

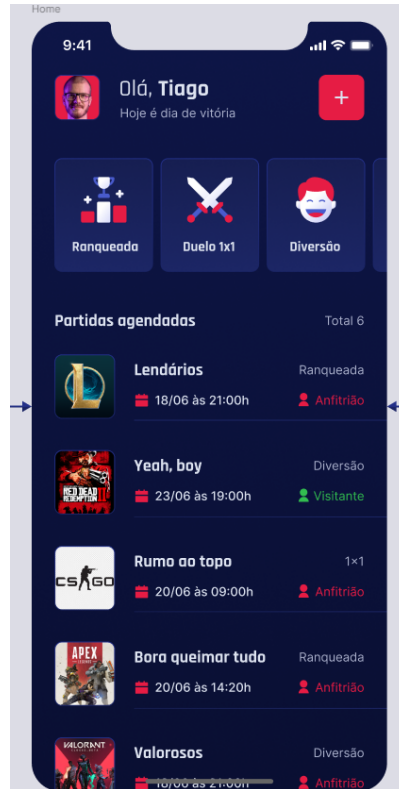
A aplicação deve demonstrar o fluxo e permitir explicar as decisões de interface e código. Não seria correto afirmar que ela agenda uma partida real: o botão informa explicitamente que nenhum agendamento foi salvo. Também não há validação completa de calendário ou de intervalo de horas.

03. Referências visuais e leitura das telas

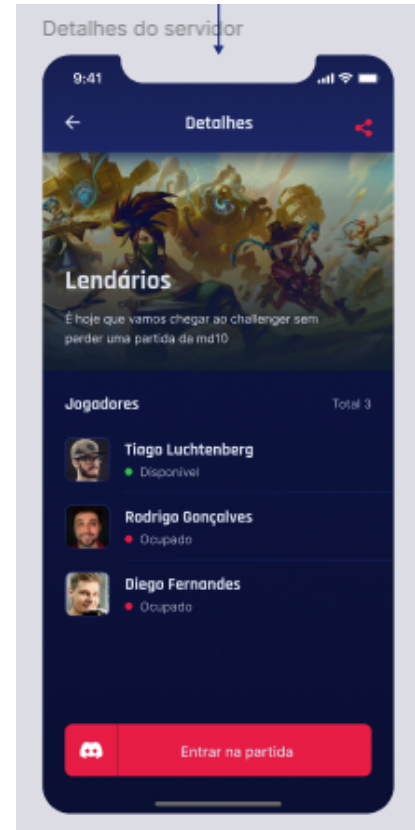
As referências definem hierarquia, cores, fontes, alinhamentos e estados. Uma tela é composta por elementos interativos reais; não foi implementada simplesmente exibindo uma captura de tela como fundo.



tela-01-login.png



tela-02-home.png



tela-2.1-home.png

Fonte: imagens locais de referencias/. São protótipos fornecidos, não capturas do aplicativo executado.

Login: a ilustração chama a atenção, o título apresenta o propósito e o botão oferece uma ação principal.

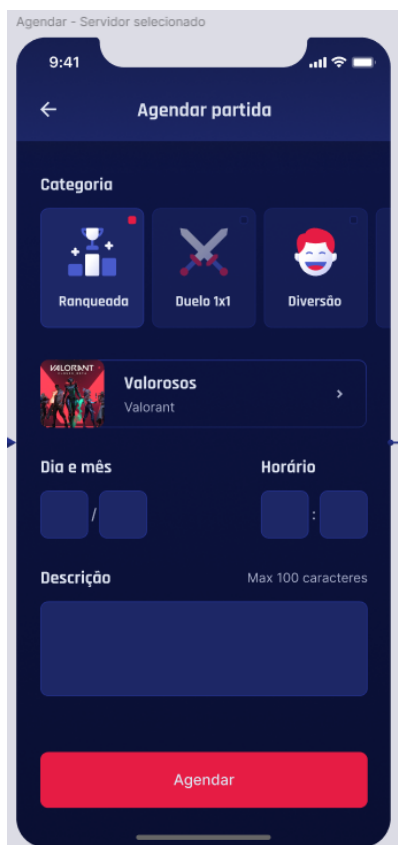
Home: avatar e saudação aparecem no topo; o botão + abre o formulário; categorias ficam em faixa horizontal; as partidas formam a lista vertical.

Detalhes: o banner apresenta o grupo, seguido da lista de jogadores e da ação de entrada.

Agendar é uma tela com vários estados



tela-03-agendar.png

tela-06-agendar-servidor
selecionado.png

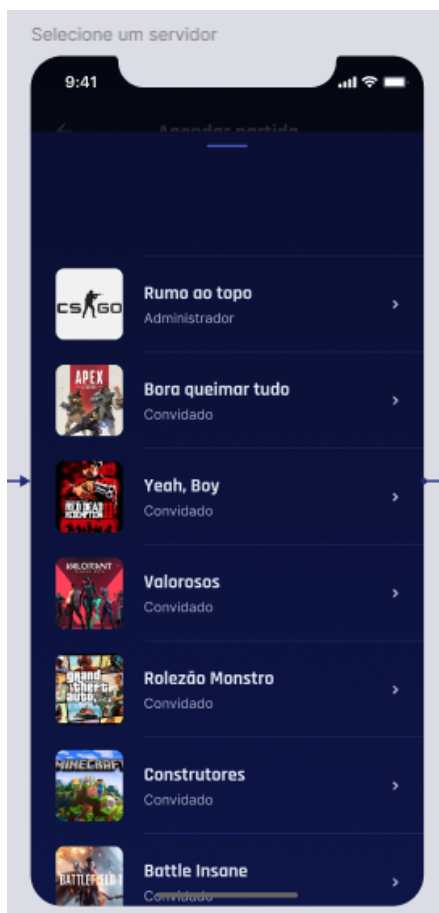
tela-07-agendar.png

Fonte: imagens locais de referências/. São protótipos fornecidos, não capturas do aplicativo executado.

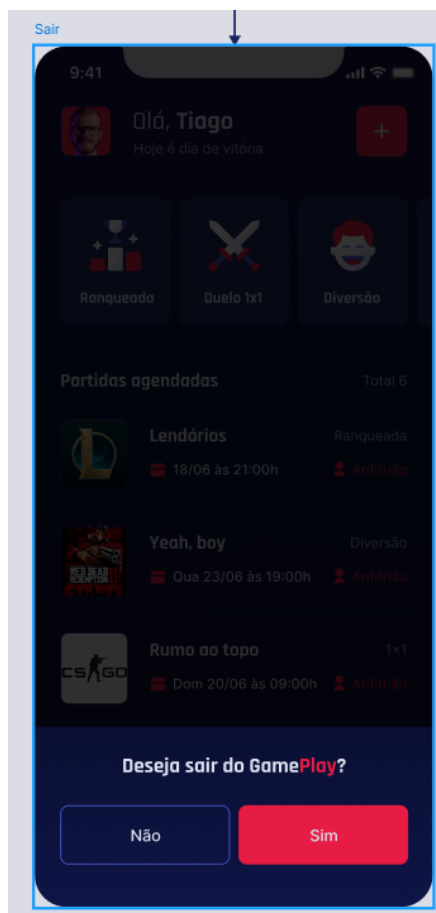
As imagens não exigem uma rota por situação. A mesma tela pode estar vazia, ter uma categoria selecionada, mostrar Valorosos e conter campos preenchidos. O teclado também altera o espaço disponível sem transformar o formulário em outra rota.

O estado atual utiliza a categoria vazia e nenhum servidor escolhido ao abrir. Depois do toque, a seleção se reflete nos mesmos componentes. No protótipo existem pequenas diferenças de rótulos entre capturas; a implementação usa **Horário** e **Max 100 caracteres**.

Os dois modais



tela-05-selecione-servidor.png



tela-08-sair.png

Fonte: imagens locais de referências/. São protótipos fornecidos, não capturas do aplicativo executado.

O seletor de servidores aparece sobre Agendar. A confirmação de saída aparece sobre a Home. Ambos são sobreposições controladas por um booleano de estado. Não é necessário criar novas páginas de navegação para esses comportamentos.

Algumas imagens locais diferem dos recortes do protótipo. Valorosos usa um logo de Valorant, e alguns jogadores usam iniciais. A lista reutiliza os seis grupos existentes; nomes e ordem não são uma reprodução integral dos grupos visíveis na tela cinco. O fundo das telas é sólido, sem reproduzir todos os efeitos de gradiente das referências.

04. Tecnologias e o papel de cada uma

O projeto usa npm e possui package-lock.json. As versões abaixo são as declaradas em package.json nesta edição. O prefixo ~ permite atualizações compatíveis dentro da faixa indicada; o lockfile registra a resolução de dependências usada na instalação.

Tecnologia	Versão declarada	Responsabilidade
Expo	~57.0.24	Ferramentas e integração para desenvolver e exportar o app
React	19.2.3	Componentes, renderização, props e hooks
React Native	0.86.3	View, Text, inputs, listas, modais e layout nativo
TypeScript	~6.0.3	Verificação estática de tipos
Expo Router	~57.0.22	Rotas por arquivos e navegação em pilha
Expo Image	~57.0.5	Exibição de imagens locais
Expo Font	~57.0.4	Carregamento das fontes locais
Expo Symbols	~57.0.3	Ícones de voltar, compartilhar e seta
Safe Area Context	~5.7.0	Ajuste às áreas reservadas pelo sistema

React e React Native não são a mesma coisa. React fornece o modelo de componentes e atualização da interface. React Native oferece componentes que representam elementos de interface nas plataformas suportadas. Expo reúne ferramentas e módulos que já estavam no projeto. Expo Router organiza como as telas são encontradas e abertas.

Biblioteca instalada não significa uso direto em todas as telas

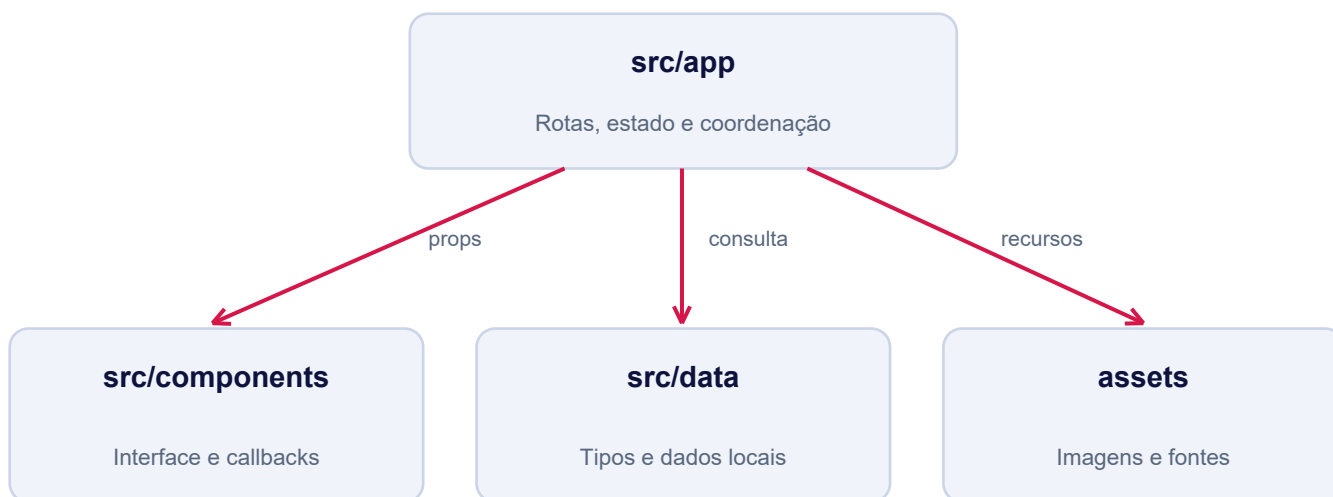
O package.json também inclui React DOM e React Native Web para web; Splash Screen e Status Bar para a abertura e a barra de status; Screens, Gesture Handler, Reanimated e Worklets; além de módulos como expo-device, expo-constants, expo-linking, expo-system-ui, expo-web-browser, expo-glass-effect e @expo/ui. Parte dessa base veio do template ou é utilizada pela infraestrutura.

Não se deve afirmar que foi programada uma animação própria com Reanimated só porque a dependência está instalada. Os modais do trabalho usam a animação fornecida pelo Modal. Nenhum pacote novo foi necessário para implementar as quatro telas e as seleções.

Por que manter essa base? Ela já resolve as necessidades do trabalho. Uma migração aumentaria o esforço e os riscos sem melhorar os conceitos que precisam ser demonstrados.

05. Arquitetura: visão geral

A organização é simples e separada por responsabilidade. Não há uma implementação formal de Clean Architecture, MVC completo ou camadas de domínio, repositório e infraestrutura. As telas coordenam estado e ações; os componentes organizam partes visuais; os dados locais representam a demonstração.



As ações dos componentes retornam à tela por callbacks.

Quem decide e quem apresenta

Uma tela, como Agendar, decide qual categoria e qual servidor estão selecionados. CategoryCard recebe os dados e a informação de seleção. O cartão não cria uma segunda seleção própria. Quando recebe um toque, chama a função passada pela tela. Isso mantém uma única origem para a informação.

Os dados em src/data podem ser importados por várias partes da interface. O módulo servers.ts atende tanto Detalhes quanto o seletor. Separar esses dados evita colocar registros extensos diretamente no JSX. Assets reúne os recursos visuais que essas telas precisam.

Por que os estilos permanecem junto dos componentes

Cada tela termina com StyleSheet.create. Assim, quem abre o arquivo encontra lógica, JSX e aparência na mesma leitura. Para o tamanho atual, isso facilita manutenção e apresentação. A paleta se repete em alguns arquivos; essa repetição é uma escolha simples, não a existência de um sistema de tema central aplicado às quatro telas.

Essa arquitetura evita criar pasta services sem serviço, pasta store sem estado global ou um formulário genérico com dezenas de props. Se o projeto crescesse, seria possível revisar essas decisões com base em necessidades reais. Não é preciso antecipar essa estrutura para explicar o aplicativo atual.

06. Pastas, arquivos e decisões de organização

Árvore focada nos arquivos do trabalho. As pastas auxiliares do template continuam presentes e são explicadas na sequência.

```
grupogame-app/  
  src/  
    app/  
      _layout.tsx  
      index.tsx  
      home.tsx  
      agendar.tsx  
      servidor/  
        [id].tsx  
    components/  
      category-card.tsx  
      appointment-card.tsx  
      screen-header.tsx  
      player-item.tsx  
      server-select-modal.tsx  
      sign-out-modal.tsx  
    data/  
      home.ts  
      servers.ts  
  assets/  
    fonts/  
    images/  
  referencias/  
  docs/  
  output/pdf/  
  package.json  
  package-lock.json  
  app.json  
  tsconfig.json
```

Por que cada pasta existe

src/app: é especial para o Expo Router. Os arquivos definem destinos de navegação. `index.tsx` representa `/`, `home.tsx` representa `/home` e `servidor/[id].tsx` representa um destino dinâmico. `_layout.tsx` configura o navegador comum. Componentes auxiliares não ficam aqui para não virarem rotas acidentalmente.

src/components: partes visuais usadas várias vezes ou que organizam um trecho importante. `CategoryCard` aparece nas categorias de Home e Agendar. `ScreenHeader` atende Detalhes e Agendar. `PlayerItem` e `AppointmentCard` se repetem em listas. Os dois modais ficam separados porque contêm sua própria árvore visual e estilos.

src/data: contém tipos e dados de demonstração. É possível trocar um nome ou examinar os IDs sem procurar dentro de uma tela. Os tipos permanecem junto dos registros porque são pequenos e diretamente relacionados; não foi necessária uma pasta `types` separada.

assets: arquivos exibidos pelo app, como imagens, ícones e fontes. **referencias:** material usado como orientação visual e o `exemplo.js` de teclado. Uma referência não vira automaticamente um recurso exibido

na interface.

docs: decisões e explicações. **output/pdf:** artefato final para estudo. O gerador da apostila fica em docs/pdf e não participa do bundle do aplicativo.

O que veio do template

src/app/explore.tsx permanece como rota de demonstração, sem botão no fluxo principal. Componentes app-tabs, animated-icon, themed-text, themed-view, ui/collapsible e arquivos de hooks e constants pertencem à base inicial. Não foram removidos apenas para “limpar” a arquitetura. O layout atual usa Stack, não as abas preservadas.

node_modules contém dependências instaladas; .expo contém arquivos de trabalho do Expo; .git contém histórico de versionamento. Essas pastas não representam novas telas ou camadas de negócio. scripts/reset-project.js pertence ao template e não é necessário para executar o trabalho.

Resposta para o professor: “Separei rotas, componentes e dados para que cada arquivo tenha uma responsabilidade fácil de encontrar. Mantive estilos junto da interface e tipos junto dos dados porque o projeto é pequeno. Evitei pastas e abstrações que ainda não teriam uso.”

07. Configuração e execução

Os quatro arquivos de configuração mais importantes

package.json informa nome, entrada, scripts e dependências. A entrada expo-router/entry inicia a integração do Router. **package-lock.json** registra a árvore resolvida de pacotes para tornar instalações mais consistentes. **app.json** configura o aplicativo Expo. **tsconfig.json** configura o compilador TypeScript.

No app.json, a orientação é portrait, a exportação web é static e os plugins incluem Router e Splash Screen. As opções typedRoutes e reactCompiler estão habilitadas. Isso não significa que todas as APIs foram otimizadas manualmente ou que o app tem autenticação; são configurações de ferramentas.

O alias dos imports

Trecho real de tsconfig.json:

```
"paths": {
  "@/*": ["../src/*"],
  "@/assets/*": ["../assets/*"]
}
```

O import de @/components/category-card aponta para src/components/category-card. Já @/assets/images/... aponta para assets/images/... na raiz, por causa do mapeamento mais específico. Isso reduz caminhos relativos longos, como várias sequências de ../. O alias facilita localização; não cria uma camada de arquitetura.

Comandos para demonstrar

```
npm.cmd start
npm.cmd run android
npm.cmd run ios
npm.cmd run web
npx.cmd tsc --noEmit --incremental false
```

Os scripts android e ios iniciam o Expo com a opção da plataforma. Executar um simulador iOS local depende de ambiente compatível; no Windows não se deve prometer abrir o simulador da Apple. Expo Go em aparelho e emuladores disponíveis dependem da configuração do ambiente e da compatibilidade com o SDK instalado.

No PowerShell deste projeto, .cmd evita o bloqueio de execução de scripts .ps1. npm start usa o script start, que chama expo start. Não execute reset-project para “abrir” o trabalho: ele é um utilitário de reinicialização da base.

Não existem pastas nativas ios e android mantidas manualmente neste projeto. As configurações do Expo são a base para gerar projetos nativos quando necessário. A exportação usada nas verificações gera bundles; ela não equivale a instalar um APK ou publicar em uma loja.

08. React, componentes e JSX

Um componente é uma função que descreve uma parte da interface. O React chama essa função durante a renderização. O retorno usa JSX, uma sintaxe que mistura elementos visuais e expressões JavaScript. Em arquivos .tsx, isso aparece junto da tipagem TypeScript.

Exemplo didático, simplificado para mostrar a composição:

```
function Saudacao({ nome }: { nome: string }) {  
  return (  
    <View>  
      <Text>Olá, {nome}</Text>  
    </View>  
  );  
}
```

View organiza os filhos; Text exibe conteúdo textual; nome é uma prop. As chaves inserem uma expressão no JSX. O componente poderia receber “Tiago” ou outro nome sem alterar sua estrutura. Nomes de componentes começam com letra maiúscula para distingui-los dos elementos especiais reconhecidos pela sintaxe.

Os componentes usados no projeto

Elemento	Papel no aplicativo
View e Text	Estrutura visual e textos
Pressable	Ação ao toque e feedback de pressionamento
ScrollView	Conteúdo rolável do Login e do formulário
FlatList	Listas de partidas, categorias, jogadores e servidores
TextInput	Entrada de dia, mês, hora, minuto e descrição
Modal	Seleção de servidor e confirmação de saída
KeyboardAvoidingView	Ajuste de espaço quando o teclado aparece
SafeAreaView	Áreas seguras, importada de safe-area-context
Image e SymbolView	Recursos visuais dos pacotes Expo já instalados

StyleSheet não é uma tag visual. É uma API que organiza objetos de estilo. useState e useEffect também não são elementos da tela: são hooks usados durante a execução de componentes.

Renderização condicional

O JSX pode incluir ou omitir um elemento conforme um valor. O projeto usa `showFeedback && (...)` para exibir a mensagem somente depois do toque. Usa um ternário para escolher entre foto e iniciais do jogador. Usa `selectedServer?.name ?? 'Selecione um servidor'` para fornecer um texto inicial quando ainda não há escolha.

A interface é uma descrição do estado atual. Em vez de procurar um botão e mudar sua cor manualmente, atualizamos o estado e deixamos o JSX calcular o próximo resultado.

09. TypeScript e sintaxe que aparece no código

TypeScript adiciona verificação estática ao JavaScript. Ele ajuda a detectar props faltando, valores incompatíveis e nomes incorretos antes da execução. Os tipos não substituem validação de dados recebidos de uma API e não garantem que uma data digitada exista no calendário.

Trecho real dos tipos:

```
export type CategoryId = 'ranked' | 'duel' | 'fun' | 'training';

type Props = {
  visible: boolean;
  selectedServerId: string | null;
  onSelect: (serverId: string) => void;
  onClose: () => void;
};
```

CategoryId é uma união de quatro valores literais. Ela restringe o conjunto de IDs aceitos. selectedServerId aceita uma string ou null. A função onSelect recebe um ID e retorna void: o chamador não espera um valor de retorno útil. onClose não recebe parâmetros.

Símbolos que você precisa reconhecer

Sintaxe	Como explicar
export e import	Tornam um valor disponível e o utilizam em outro módulo
import type	Importa somente um tipo para a verificação estática
avatar?: ...	A propriedade é opcional
Category[]	Um array de categorias
const { id } = ...	Desestrutura uma propriedade do objeto retornado
const [valor, setter] = ...	Desestrutura o par retornado pelo hook
===	Compara valor sem conversão automática de tipo
?.	Acessa ou chama apenas se o valor não for nulo/definido
??	Fornece alternativa para null ou undefined
condicao ? a : b	Escolhe entre dois resultados
() => ...	Cria uma função, frequentemente usada como callback

Uma arrow function não é executada só por ser criada. Em `onPress={() => handleSelectCategory(item.id)}`, a expressão define uma função que será chamada depois, no toque. Se fosse `onPress={handleSelectCategory(item.id)}`, a função seria chamada durante a renderização e seu resultado seria passado como prop. Esse não é o fluxo pretendido.

O projeto usa `strict: true`. Isso obriga o código a considerar situações como `null` e `undefined`. O estado inicial sem servidor e o tratamento de servidor não encontrado são exemplos em que a tipagem orienta uma interface mais explícita.

10. Props, callbacks, estado e dados derivados

Props são entradas de um componente. Estado é informação mantida por uma instância de componente ao longo das renderizações. Um callback é uma função passada como prop para comunicar uma ação. Dados derivados são valores calculados a partir de props, estado ou registros existentes.

Exemplo central: CategoryCard

Trecho de uso em Agendar, com a dica de acessibilidade omitida para facilitar a leitura:

```
<CategoryCard
  category={category}
  selected={selectedCategoryId === category.id}
  onPress={() => setSelectedCategoryId(category.id)}
  showSelectionIndicator
  dimUnselected
/>
```

category contém o nome e o ícone. selected é um booleano calculado. onPress é a função que atualiza o estado da tela. As props sem valor explícito, como dimUnselected, equivalem a passar true. CategoryCard não guarda outra cópia da seleção em useState.

O ciclo de atualização

1. A tela renderiza com selectedCategoryId igual a null.
2. Todos os cartões recebem selected=false.
3. O usuário toca em Ranqueada e o callback solicita a atualização para 'ranked'.
4. O React processa o estado e renderiza a tela novamente.
5. A comparação dá true somente para Ranqueada e seu estilo muda.

O setter não altera imediatamente a variável da renderização que já está em execução. Cada renderização lê seu próprio valor de estado. Quando a próxima atualização depende do valor anterior, a forma funcional é útil: setSelectedCategoryId(current => ...).

Evitar estado desnecessário

visibleAppointments é calculado com filter; o total vem de visibleAppointments.length. selectedServer vem de servers.find. Texto e cor do papel do jogador são calculados a partir de props. Não há necessidade de criar um setter para cada uma dessas informações.

Se o total fosse guardado separadamente, seria necessário lembrar de atualizá-lo sempre que o filtro mudasse. Calculá-lo da lista elimina essa possibilidade de divergência. Esse é o motivo concreto para usar dados derivados, e não apenas uma preferência de estilo.

Onde o estado vive

Home mantém filtro e abertura da saída. Agendar mantém categoria, servidor, modal, campos e feedback. Detalhes mantém a mensagem de feedback. Login não precisa de estado local. Os modais recebem o estado da tela que os controla. Não foi necessário Context, Redux ou outro gerenciador global.

11. Dados simulados e relações entre IDs

`src/data/home.ts` reúne usuário, categorias e partidas. `src/data/servers.ts` reúne servidores e jogadores. Esses dados são constantes locais importadas pelas telas. Não há fetch, requisição HTTP ou serviço remoto alimentando as listas.

Tipo	Identificação e campos	Onde é usado
Category	id, title, matchLabel, icon	Home e Agendar
Appointment	id, serverId, categoryId, título, data, hora, imagem, isHost	Cards da Home
Server	id, name, game, image, banner, description, isAdmin, players	Detalhes e seletor
Player	id, name, initials, avatar opcional, status	Lista de jogadores

Uma partida possui seu próprio id e um `serverId`. O primeiro identifica a linha da lista; o segundo aponta para o servidor. Não são a mesma responsabilidade. Várias partidas poderiam apontar para um mesmo servidor, ainda que os dados atuais sejam pequenos.

Ligação concreta

A partida `match-5` contém `serverId`: `'valorosos'`. O registro do servidor tem id: `'valorosos'`. Quando o card envia esse identificador para a rota, Detalhes encontra o grupo com `find`. Em Agendar, o mesmo ID identifica a escolha, mas não é necessário navegar para Detalhes para selecioná-lo.

Trecho real da busca em Agendar:

```
const selectedServer = servers.find(
  (server) => server.id === selectedServerId
);
```

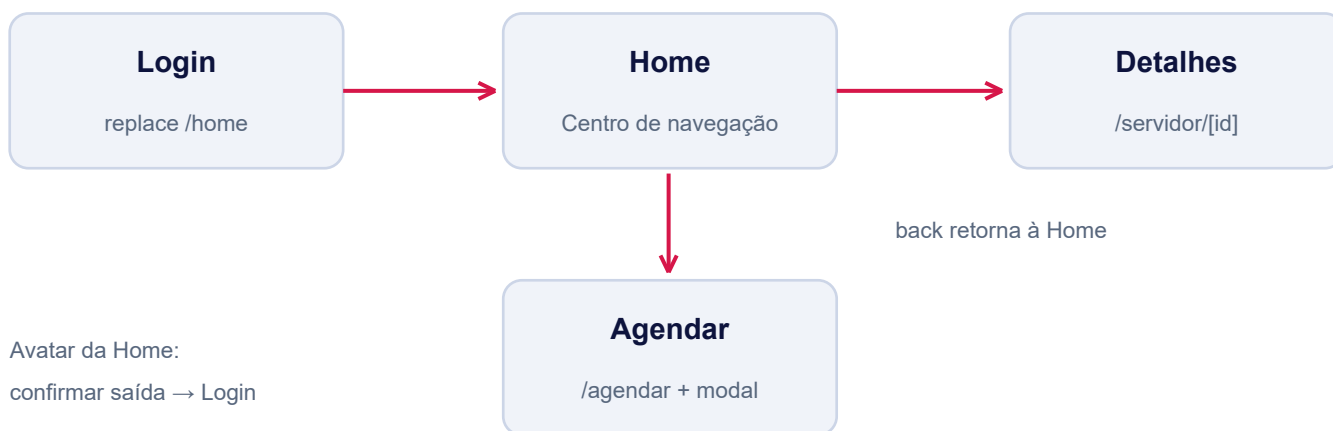
`find` retorna um elemento ou `undefined`. `filter` retorna um novo array com os elementos que satisfazem a condição. `map` transforma cada elemento e retorna outro array; por isso é usado para montar cartões e gerar parâmetros de rotas estáticas. Nenhuma dessas três operações precisa modificar o array original.

Simulação e consistência

O mesmo grupo fictício de três jogadores é compartilhado entre os servidores. Os status não são presença online real. `isAdmin` é o papel do usuário no grupo; `isHost` é o papel em uma partida. Uma pessoa pode ser anfitriã de uma partida sem que isso signifique administrar todo o servidor.

O sexto grupo e algumas imagens são alternativas de demonstração. A origem dos assets está registrada em `assets/README.md`. Esses registros não significam que o aplicativo acessou contas ou recebeu permissão de usuários reais do Discord.

12. Navegação: rotas, pilha e parâmetros



O Expo Router associa arquivos a caminhos. O Stack do layout raiz organiza as telas em uma pilha. Abrir uma tela pode colocar um destino sobre o anterior; voltar pode retirar a tela atual e revelar a anterior. Um modal de React Native, por sua vez, não adiciona uma rota a essa pilha.

Arquivo	Caminho	Função
src/app/index.tsx	/	Login
src/app/home.tsx	/home	Lista principal
src/app/agendar.tsx	/agendar	Formulário
src/app/servidor/[id].tsx	/servidor/valerosos, por exemplo	Detalhes por ID

Métodos usados

replace: troca a rota atual. No Login, replace('/home') evita deixar aquela instância do Login abaixo da Home. A saída também substitui Home por /. Isso é navegação, não uma proteção de autenticação.

navigate: abre ou retorna a um destino conforme o estado da pilha. A Home usa navigate para Detalhes e Agendar. Na revisão, foi preferido a push para evitar empilhar repetidamente o mesmo destino ativo em toques duplicados. Isso não é uma afirmação de que qualquer combinação possível de rotas nunca produzirá outra instância.

back: volta na pilha. handleBack verifica canGoBack; quando não há histórico, como em acesso direto, replace('/home') oferece um destino de retorno.

Passar um ID, não um objeto inteiro

```
router.navigate({
  pathname: '/servidor/[id]',
  params: { id: serverId },
});
```

Detalhes lê o parâmetro por `useLocalSearchParams` e consulta os dados locais. A URL fica curta, os dados têm uma origem definida e o destino pode ser acessado diretamente pelo identificador. Se não houver correspondência, a tela mostra “Servidor não encontrado”. O parâmetro não é convertido em um servidor confiável apenas por estar tipado.

`generateStaticParams` retorna os IDs conhecidos para gerar as páginas na exportação estática web. Essa função atende ao processo de exportação; o usuário não a chama ao tocar em uma partida. A rota de `template /explore` continua existente, embora não faça parte do fluxo principal.

13. Inicialização, fontes e tela de Login

O que acontece antes da primeira tela

_layout.tsx chama preventAutoHideAsync fora do componente para manter a abertura enquanto as fontes são preparadas. useFonts carrega RajdhaniBold e Inter de arquivos locais. useEffect observa fontsLoaded e fontError; quando uma dessas condições libera a espera, oculta a abertura. Se ainda estiver carregando, o componente retorna null.

O array de dependências de useEffect informa quais valores provocam nova execução do efeito após a renderização. Esse hook é adequado aqui porque esconder a abertura é um efeito externo à descrição do JSX. Não seria necessário usar useEffect só para calcular o nome do servidor a partir de um ID.

O layout retorna um Fragment com StatusBar e Stack. Fragment agrupa elementos sem criar uma View extra. headerShown: false evita um cabeçalho nativo adicional sobre os cabeçalhos desenhados pelo projeto.

Login passo a passo

1. SafeAreaView protege o conteúdo das áreas do sistema.
2. ScrollView permite rolar caso a altura seja insuficiente.
3. Image exibe a ilustração local com contentFit="contain".
4. View agrupa título, descrição e botão.
5. Pressable chama handleSignIn, que substitui a rota pela Home.

Trecho real da ação, sem comentários:

```
function handleSignIn() {  
  router.replace('/home');  
}
```

Não há usuário e senha, token, OAuth ou consulta ao Discord nessa função. O nome do botão pertence ao protótipo, mas a ação é uma simulação acadêmica. A tela não usa useState porque não há um valor local editável ou selecionável que ela precise lembrar.

Decisões visuais

A ilustração mantém a proporção com aspectRatio. O contêiner possui largura máxima para não esticar indefinidamente na web. marginTop negativo aproxima o texto da imagem, acompanhando a composição da referência. O botão usa row para colocar ícone e legenda lado a lado; o texto usa flex: 1 para preencher o espaço restante.

Pressable fornece pressed à função de style. O array aplica styles.buttonPressed durante o toque, reduzindo a opacidade. Isso é um feedback temporário de interação e não um estado de autenticação.

14. Home: listas, filtro e composição

A Home é a tela de consulta. Ela combina perfil fictício, ação de agendar, categorias e partidas. Os dados vêm de `src/data/home.ts`. O estado `selectedCategoryId` controla o filtro; `showSignOut` controla o modal de saída. Uma escolha não interfere na outra.

Como a lista é calculada

Trecho real:

```
const visibleAppointments = selectedCategoryId
  ? appointments.filter(
      (appointment) => appointment.categoryId === selectedCategoryId
    )
  : appointments;
```

Sem seleção, são exibidas todas as partidas. Com seleção, `filter` produz as correspondentes. O total usa `visibleAppointments.length`, por isso acompanha o resultado. A mensagem de lista vazia fica em `ListEmptyComponent`.

Como o toque alterna o filtro

```
setSelectedCategoryId(
  (current) => current === categoryId ? null : categoryId
);
```

Se a categoria tocada já for a atual, o estado volta a `null`. Se for outra, o ID muda. Essa regra pertence à Home. Em Agendar, tocar novamente na categoria atual mantém a seleção, pois a função da tela é escolher uma opção para o formulário, não alternar um filtro.

Duas direções de rolagem

A `FlatList` vertical recebe `visibleAppointments` e renderiza `AppointmentCard`. O topo está em `ListHeaderComponent`, então saudação e categorias acompanham a rolagem principal. Dentro do cabeçalho há uma `FlatList` horizontal para as categorias. As direções distintas atendem ao protótipo.

`keyExtractor` retorna IDs estáveis. O React consegue relacionar os itens anteriores aos novos quando a lista muda. Usar o índice como identificação seria menos adequado em uma lista que pode ser filtrada. `extraData={selectedCategoryId}` faz a lista horizontal considerar a mudança visual de seleção mesmo mantendo o array `categories`.

Delegar a ação do card

`AppointmentCard` recebe a partida, o rótulo da categoria e `handleOpenServer`. Ao toque, entrega `appointment.serverId`. A Home decide usar o Router. Essa separação permite explicar o card como interface e a tela como coordenadora da navegação.

O avatar abre a confirmação de saída. O botão + chama `navigate('/agendar')`. Nenhuma dessas ações precisa editar o array original de partidas.

15. Componentes reutilizáveis e suas props

Reutilização não significa transformar todos os elementos em componentes genéricos. O projeto separa elementos repetidos ou suficientemente grandes para tornar a tela mais legível. Cada componente tem poucas entradas e uma responsabilidade identificável.

Componente	Entradas principais	Por que foi separado
CategoryCard	category, selected, onPress e opções visuais	Repetido em duas telas e em várias categorias
AppointmentCard	appointment, categoryLabel, onPress opcional	Organiza cada partida da Home
ScreenHeader	title, onBack, onShare opcional	Reutiliza título e ações em duas telas
PlayerItem	player	Padroniza as linhas da lista de jogadores
ServerSelectModal	visible, selectedServerId, onSelect, onClose	Organiza a lista e a sobreposição
SignOutModal	visible, onCancel, onConfirm	Isola a confirmação sem aumentar a Home

CategoryCard: mesma interface, regras diferentes

O cartão recebe selected, em vez de calcular sozinho a escolha do formulário. Isso evita que cada cartão mantenha um booleano independente e mais de um fique marcado. O callback permite que Home e Agendar tenham comportamentos distintos usando a mesma aparência base.

showSelectionIndicator e dimUnselected começam em false. A Home não precisa fornecê-los e mantém a aparência original. Agendar ativa ambos. accessibilityHint também é opcional e possui texto padrão apropriado ao filtro da Home; Agendar passa uma instrução de seleção única.

Props opcionais na prática

ScreenHeader só mostra compartilhar quando recebe onShare. Caso contrário, uma View vazia reserva a largura do botão, mantendo o título centralizado. O componente não precisa saber se está em Agendar ou em Detalhes: a presença do callback define a ação disponível.

AppointmentCard fica desabilitado se onPress não existir. A chamada onPress?.(appointment.serverId) usa encadeamento opcional. Na Home atual o callback é fornecido, então o card navega normalmente.

O que não foi extraído

Os campos numéricos compartilham estilo, mas não viraram um componente universal de formulário. Os botões do Login e de Detalhes reutilizam o ícone, porém continuam em suas telas. A semelhança pode justificar uma extração futura; não foi necessário reestruturar uma tela funcional só para eliminar poucas linhas repetidas.

16. Detalhes do servidor

O arquivo `servidor/[id].tsx` representa uma tela capaz de mostrar qualquer servidor local conhecido. Não existe um arquivo separado para Valorosos, Lendários e os demais grupos. O parâmetro escolhe os dados que alimentam a mesma estrutura de JSX.

Leitura e proteção contra ausência de dados

Trecho real:

```
const { id } = useLocalSearchParams<{ id?: string | string[] }>();
const server = servers.find((item) => item.id === id);
```

Se o ID estiver ausente, inválido ou não corresponder a uma string esperada, `find` não encontra registro. O retorno antecipado `if (!server)` exibe um aviso e o cabeçalho de voltar. Isso evita acessar `server.players` quando `server` não existe.

A lista de jogadores

`FlatList` recebe `server.players`. Seu cabeçalho contém banner, nome, descrição e a linha “Jogadores / Total”. Cada registro é passado a `PlayerItem`, que decide mostrar avatar ou iniciais. O texto e a cor do status vêm de `player.status`, não de um estado de presença atualizado pela rede.

O rodapé com a ação principal fica fora da lista e a lista recebe flex: 1. Isso destina a ela o espaço entre o cabeçalho e o rodapé. O banner tem imagem absoluta cobrindo sua área e um fundo translúcido sob os textos para favorecer a leitura.

Entrar e compartilhar

`handleJoin` atualiza `feedback` com o ID atual e uma mensagem de entrada simulada. Ao renderizar, a tela só mostra a mensagem se `feedback.serverId` corresponder ao servidor atual. Assim, uma mensagem produzida para outro grupo não aparece como se fosse desta tela.

`handleShare` usa `async/await` para aguardar `Share.share`. O sistema recebe nome e descrição. Não há link de convite do Discord. Um cancelamento identificado como `AbortError` é ignorado; outras falhas mostram uma mensagem local de indisponibilidade. A experiência de compartilhamento depende da plataforma.

`handleBack` usa o histórico da pilha, com retorno para Home quando não houver histórico. O cabeçalho apenas dispara esse callback. Esse é um exemplo de componente visual reutilizável sem responsabilidade própria sobre rotas.

17. Agendar: uma tela, vários estados

Agendar usa uma única rota para todas as situações do protótipo. O formulário não cria novas telas quando uma categoria é selecionada, quando o teclado abre ou quando um servidor é escolhido. São mudanças de estado e de espaço disponível na mesma interface.

Estado	Valor inicial	O que controla
selectedCategoryId	null	Uma categoria escolhida
selectedServerId	null	Um grupo escolhido
showServerSelect	false	Visibilidade do modal de grupos
day, month, hour, minute	String vazia	Valores dos campos numéricos
description	String vazia	Texto de até 100 caracteres
showFeedback	false	Mensagem após tocar em Agendar

selectedServer não aparece como outro useState porque é derivado do ID com find. Essa escolha mantém os dados do grupo em um único lugar. O booleano do modal não é o mesmo que a escolha: é possível ter um grupo escolhido com o modal fechado ou aberto.

Exclusividade da categoria

Um único ID já impõe que só um cartão compare igual ao estado atual. Guardar quatro booleanos exigiria desmarcar os outros manualmente a cada toque. A implementação evita essa coordenação adicional.

Trecho real do array de estilos em CategoryCard:

```
[
  styles.card,
  selected && styles.selected,
  showSelectionIndicator && selected && styles.selectedWithIndicator,
  pressed && styles.pressed,
  dimUnselected && !selected && styles.unselected,
]
```

styles.unselected define opacity: 0.4. Um cartão não escolhido fica apagado, inclusive ícone e texto. Ao selecionar, essa condição se torna falsa, deixando de aplicar a baixa opacidade. O cartão anterior recebe novamente o estilo. A regra aparece depois do feedback de pressionamento para não clarear indevidamente uma opção ainda não selecionada durante o toque.

showSelectionIndicator exibe o quadrado no canto. Ele fica vermelho quando selected é true. O estilo selectedWithIndicator mantém a borda azul em Agendar. Na Home, a seleção continua com borda vermelha e sem as opções visuais adicionais.

Opacidade não significa desabilitado. Um cartão apagado continua recebendo toques. A seleção é uma regra de estado; o destaque é uma consequência dessa regra no estilo.

18. TextInput e formulário controlado

Um campo controlado recebe seu texto do estado e informa alterações por callback. O usuário digita, `onChangeText` recebe o novo texto, o setter atualiza o estado e a renderização devolve o valor para o componente. O dado não fica oculto apenas dentro do input.

Trecho de Agendar, com props visuais omitidas:

```
const [day, setDay] = useState('');

<TextInput
  value={day}
  onChangeText={({text}) => setDay(text.replace(/\D/g, ''))}
  keyboardType="number-pad"
  maxLength={2}
/>
```

Por que o número é uma string

`TextInput` trabalha com texto. A string permite representar o campo vazio e um mês como “06” sem perder o zero. Transformar tudo em número durante a digitação dificultaria esses estados intermediários. Uma aplicação com agendamento real poderia converter e validar os valores no momento apropriado.

Na expressão regular, `\D` representa um caractere que não é dígito. A flag `g` aplica a substituição a todas as ocorrências. `replace(..., "")` remove esses caracteres. Isso é útil inclusive ao colar texto ou digitar pela web. `keyboardType` solicita um teclado adequado, mas sozinho não garante a validade do valor.

Descrição

`description` é uma string separada. O input recebe `value={description}`, `onChangeText={setDescription}`, `multiline` e `maxLength={100}`. Como o setter aceita o novo texto, pode ser passado diretamente. `textAlignVertical: 'top'` posiciona o conteúdo no início da área de escrita.

Limites de entrada não são validação de negócio

Dois dígitos não garantem que “99” seja um mês válido. `maxLength` de 100 não verifica se a descrição é útil. O projeto não impede um envio com campos vazios e não calcula datas futuras. O botão exibe apenas a mensagem de demonstração.

Quando Agendar é desmontada e depois aberta novamente, os estados começam nos valores iniciais. Abrir e fechar o modal não desmonta o formulário; por isso não apaga os campos. Não existe persistência após fechamento do app ou recarregamento.

19. Teclado e rolagem: o exemplo.js aplicado

O arquivo `referencias/exemplo.js` foi fornecido como referência de comportamento. Ele envolve o conteúdo com `KeyboardAvoidingView`, usa `ScrollView` e fecha o teclado por `TouchableWithoutFeedback` chamando `Keyboard.dismiss`. O projeto adaptou esses conceitos ao formulário existente, sem copiar funções de perfil, haptics, senha ou salvamento que não fazem parte do trabalho.

Três necessidades distintas

Necessidade	Recurso	O que resolve
Ajustar espaço com teclado	<code>KeyboardAvoidingView</code>	Compensa a sobreposição do teclado
Alcançar conteúdo maior	<code>ScrollView</code>	Permite deslocar o formulário
Fechar teclado por ação	<code>Keyboard.dismiss</code>	Dispensa o teclado e remove foco

`KeyboardAvoidingView` usa `behavior="padding"` no Android e iOS e fica desativado na web. A intenção é ajustar o espaço inferior quando o teclado se sobrepõe à tela. Isso depende do comportamento da plataforma; a compilação não garante que todos os campos fiquem confortáveis em todos os aparelhos.

Estrutura simplificada, equivalente à organização atual

```
<KeyboardAvoidingView behavior="padding">
  <SafeAreaView>
    <View>
      <ScreenHeader title="Agendar partida" onBack={handleBack} />
      <ScrollView keyboardShouldPersistTaps="handled">
        <TouchableWithoutFeedback onPress={fecharTeclado}>
          <View>{/* categorias, servidor, campos e botão */}</View>
        </TouchableWithoutFeedback>
      </ScrollView>
    </View>
  </SafeAreaView>
</KeyboardAvoidingView>
```

Este trecho omite estilos e outras props para mostrar a hierarquia. O código completo está no apêndice. A View interna é o filho único de `TouchableWithoutFeedback`; `accessible={false}` evita agrupar o formulário inteiro como um único elemento acessível.

Props e decisões

`keyboardShouldPersistTaps="handled"` permite que botões tratem o toque com o teclado aberto. `keyboardDismissMode` usa `interactive` no iOS e `on-drag` nas demais plataformas. `bounces={false}` desativa o efeito elástico onde suportado. `flexGrow: 1` permite preencher a altura disponível sem impedir que o conteúdo cresça e role.

`fecharTeclado` chama `Keyboard.dismiss`. A mesma função é usada ao tocar em espaço livre, abrir a seleção de servidor e acionar Agendar. Ela não altera `day` ou `description`. O botão está dentro da rolagem, o que

ajuda a alcançá-lo em uma área reduzida.

O que dizer: “Eu não movo cada campo manualmente. O contêiner ajusta o espaço com o teclado, a rolagem permite alcançar os elementos e a função fecharTeclado apenas dispensa o teclado. Os valores continuam no estado.”

20. Seleção de servidor: tela cinco

O bloco abaixo das categorias começa com “Selecione um servidor”, área de imagem vazia e seta. Ele é um Pressable. Quando tocado, fecha o teclado e muda showServerSelect para true. O ServerSelectModal aparece sobre Agendar, mantendo o formulário montado.

Contrato entre tela e modal

Trecho real das funções da tela, sem comentários:

```
function handleOpenServers() {  
  fecharTeclado();  
  setShowServerSelect(true);  
}  
  
function handleSelectServer(serverId: string) {  
  setSelectedServerId(serverId);  
  setShowServerSelect(false);  
}
```

O modal recebe visible, selectedServerId, onSelect e onClose. Ele não tem seu próprio useState de seleção. Cada linha chama onSelect(item.id), e a função acima atualiza o formulário. O fluxo é um callback entre componentes, não uma navegação de ida e volta com objetos na URL.

Como a lista funciona

FlatList usa servers como data, server.id como chave e selectedServerId em extraData. A indicação de seleção para leitores de tela é calculada por comparação de IDs. Cada Pressable contém a miniatura, o nome, o papel fictício no grupo e a seta.

O painel ocupa 88% da altura disponível. Sua lista pode rolar. Um fundo escuro mantém a tela anterior perceptível. O topo possui um traço que também funciona como área de fechamento por toque. Não existe implementação de gesto de arrastar o painel.

Fechamento e preservação

Escolher um item fecha e atualiza a seleção. Tocar no fundo ou no traço fecha sem mudar o ID. onRequestClose trata o botão voltar do Android enquanto o modal está aberto. Nenhuma dessas ações limpa os campos ou troca a categoria.

Ao reabrir, o servidor escolhido continua conhecido. Ao escolher outro, o único ID é substituído, e o bloco usa find para exibir os novos dados. O formulário não está salvando uma relação no banco; está apenas representando uma escolha temporária.

21. Confirmação de saída

A Home possui `showSignOut`, inicialmente `false`. Tocar no avatar muda o valor para `true` e exibe `SignOutModal`. Cancelar chama `onCancel`, que fecha o modal. Confirmar chama `onConfirm`, associado a `handleSignOut` da Home.

Trecho real:

```
function handleSignOut() {  
  setShowSignOut(false);  
  router.replace('/');  
}
```

O modal é uma apresentação da decisão. Ele não importa o Router nem conhece detalhes da rota de Login. A Home mantém essa responsabilidade. Isso torna a separação fácil de explicar: o componente informa a intenção, e a tela decide a consequência.

Por que não foi criada uma rota /sair

A confirmação é temporária e depende da tela atual. Ela precisa aparecer sobre a Home e permitir cancelar mantendo o filtro. Um Modal controlado por estado é suficiente e evita criar uma etapa de navegação para uma pergunta simples.

`transparent` e o fundo escuro permitem perceber a Home atrás do painel. `animationType="fade"` fornece a transição visual. `onRequestClose` usa o cancelamento quando o botão voltar do Android é acionado. Tocar fora também cancela. Os botões Não e Sim dividem a largura por flex: 1 em um contêiner row.

Saída simulada não é segurança de sessão

Não há token para revogar, credenciais para apagar ou servidor para notificar. Trocar a rota pela tela inicial demonstra a saída visual. Como não existe autenticação nem proteção de rotas, abrir `/home` diretamente não é bloqueado por uma sessão.

Resposta para o professor: “A confirmação usa estado e callbacks. Ao confirmar, a Home substitui sua rota pelo Login. É uma simulação do fluxo; autenticação, proteção de rotas e encerramento de sessão real estão fora do projeto.”

22. StyleSheet, Flexbox e adaptação de layout

StyleSheet.create organiza objetos nomeados. O JSX aplica esses objetos por style. O React Native aceita arrays de estilos; propriedades posteriores sobrescrevem anteriores. O projeto usa isso para feedback de toque, categoria selecionada e opacidade de opções não escolhidas.

Eixos do Flexbox

O padrão dos contêineres é coluna. Nesse caso, justifyContent distribui no eixo vertical e alignItems no horizontal. Com flexDirection: 'row', o eixo principal passa a ser horizontal. Não é correto memorizar que justifyContent é sempre horizontal: ele depende da direção.

Exemplo didático equivalente ao cabeçalho da Home:

```
header: {  
  flexDirection: 'row',  
  alignItems: 'center',  
  gap: 20,  
},  
greeting: { flex: 1 },
```

O avatar e o botão têm dimensões definidas. greeting recebe o espaço restante. gap define a distância entre filhos. Em outras partes, justifyContent: 'space-between' separa título e total ou os dois grupos de data e horário.

Propriedades que aparecem com frequência

Propriedade	Interpretação no projeto
flex: 1	Ocupa uma parcela flexível do espaço disponível
flexGrow: 1	Permite ao conteúdo crescer e preencher espaço livre
flexShrink: 0	Evita que a faixa de categorias seja comprimida
flexWrap: 'wrap'	Permite outra linha quando falta largura
padding	Espaço interno do contêiner
margin	Espaço externo ao elemento
marginTop: 'auto'	Usa espaço livre acima do rodapé quando disponível
maxWidth	Limita expansão em telas largas
minHeight	Define altura mínima, permitindo crescer com conteúdo
borderRadius	Arredonda os cantos
position: 'absolute'	Posiciona em relação à área do contêiner sem ocupar fluxo normal

Tamanho, cor e imagem

Os números de layout não são pixels físicos fixos: são unidades lógicas. `width: '100%'` usa uma proporção da largura disponível. `aspectRatio` preserva uma relação entre largura e altura. `contentFit="contain"` mostra a imagem inteira; `cover` preenche a área, podendo recortar bordas.

A paleta usa fundo #0D133D, áreas #1B245F, bordas #243189, textos #DDE3F0 e destaque #E91446. `RajdhaniBold` aparece nos títulos e `Inter` nos textos. O uso consistente ajuda a reconhecer hierarquia visual.

Limitações de responsividade

Rolagem, quebra de linha e larguras máximas ajudam, mas não substituem teste em aparelho pequeno, texto ampliado, teclado aberto e áreas seguras distintas. Não houve reprodução de todos os efeitos visuais do protótipo. Esses limites precisam ser descritos com honestidade na apresentação.

23. Acessibilidade, assets e cuidados de interface

Acessibilidade faz parte do significado dos componentes. Um botão visual precisa comunicar sua função, e a seleção de uma categoria não deve depender apenas de uma cor para quem usa leitor de tela.

Props usadas

`accessibilityRole="button"` identifica ações. `accessibilityLabel` fornece um nome compreensível. `accessibilityHint` explica a consequência do toque. `accessibilityState` informa `selected` ou `disabled`. `accessibilityLiveRegion="polite"` é usado nas mensagens, com suporte que depende da plataforma. `accessibilityViewIsModal` indica a sobreposição para tecnologias assistivas onde suportado.

Imagens decorativas recebem `accessible={false}` para não repetir informação que já aparece no texto. Um card de partida reúne em seu rótulo nome, categoria, data, horário e papel. `PlayerItem` também exibe “Disponível” ou “Ocupado”, além do ponto colorido.

Isso não significa que a aplicação passou por uma auditoria completa de acessibilidade. Leitor de tela, ordem de foco, contraste e texto ampliado continuam itens para verificação prática.

Recursos locais

As imagens são carregadas por `require` com caminhos estáticos. Isso permite ao empacotador localizar os assets. As fontes são carregadas uma vez no layout raiz e usadas pelo nome registrado. Não há download de avatar do Discord em tempo de execução.

Os materiais vieram das referências e de recursos educacionais e públicos registrados em `assets/README.md`. A autoria das marcas, personagens e artes permanece com seus titulares. O projeto é uma demonstração acadêmica; usar uma imagem local não transfere sua autoria.

Cuidar do que a interface promete

“Entrar com Discord” acompanha a referência, mas a dica e a documentação esclarecem a simulação. “Agendar” não exibe uma confirmação falsa de salvamento: informa que nenhum agendamento foi salvo. Compartilhar envia texto ao sistema, sem afirmar que criou um convite.

As categorias apagadas não ficam `disabled`. Desativá-las impediria justamente a ação de escolhê-las. Esse é um exemplo importante da diferença entre aparência e comportamento.

24. Decisões técnicas e alternativas

Uma boa defesa do projeto relaciona cada escolha a uma necessidade. A solução atual foi dimensionada para quatro telas, poucos dados e uma apresentação acadêmica.

Escolha	Motivo concreto	Quando reavaliar
Estado local	Cada formulário/filtro pertence a uma tela	Dados compartilhados e mutáveis entre muitas telas
ID na rota	Endereço simples e busca em fonte definida	Integração real ainda pode preservar o mesmo padrão
Modal para grupo	Seleção temporária mantém formulário montado	Fluxo de seleção grande e independente
Dados em src/data	Clareza e reutilização sem backend	Quando existir API e necessidade de carregamento
Estilos no arquivo	Leitura direta de cada tela	Repetição visual extensa ou tema dinâmico real
Sem biblioteca de formulário	Cinco campos simples com estado local	Formulários complexos e muitas regras
Componentes pequenos	Repetição real e organização	Novos usos demonstrarem uma abstração comum

Exemplo didático: uma escolha versus vários booleanos

Uma alternativa seria guardar `rankedSelected`, `duelSelected`, `funSelected` e `trainingSelected`. Cada toque precisaria marcar um e desmarcar três. É possível implementar corretamente, mas cria combinações inválidas, como dois valores `true`.

```
// Padrão adotado: uma única informação representa a escolha.
const [selectedCategoryId, setSelectedCategoryId] =
  useState<CategoryId | null>(null);
```

Exemplo didático: cálculo em vez de sincronização

```
// O projeto calcula o total do resultado atual.
const total = visibleAppointments.length;
```

Esse exemplo dá um nome ao cálculo para explicá-lo; no JSX real, a expressão é usada diretamente. Não existe um `useState` chamado `total` na `Home`. Criá-lo exigiria sincronizá-lo com o filtro, sem benefício neste caso.

Exemplo didático: callback em vez de dependência de navegação

Um AppointmentCard poderia importar router e navegar sozinho. O projeto preferiu receber onPress, porque a tela já coordena seus destinos. O card comunica um ID, e a Home escolhe o comportamento. Isso reduz a responsabilidade do componente sem criar um sistema de eventos complexo.

Não foi criado backend, armazenamento, autenticação ou validação real. Essas seriam evoluções de escopo, não funcionalidades escondidas em bibliotecas existentes. As decisões atuais precisam ser julgadas pelo objetivo acadêmico que atendem.

25. Verificações, limites e demonstração manual

O que foi verificado tecnicamente

O histórico registra execução de TypeScript com noEmit e verificações de variáveis/parâmetros não usados. Após a seleção de servidor, a exportação Expo gerou bundles Android, iOS e web e 13 rotas estáticas. Isso inclui rotas da infraestrutura e do template, além das telas principais.

Na etapa de comentários, o JavaScript gerado sem comentários foi comparado antes e depois nos 13 arquivos principais. As saídas permaneceram idênticas. Esse resultado confirma que a documentação no código não mudou o código executável gerado.

ESLint foi tentado, mas não está configurado. A instalação foi recusada conforme a decisão de não configurá-lo por enquanto. Não se deve registrar “lint passou”. A exportação anterior mostrou aviso relacionado às variáveis de cor do ambiente, não uma comprovação de ausência de warnings em toda execução nativa.

O que a compilação não prova

- Não prova fidelidade visual pixel a pixel ao protótipo.
- Não prova que o teclado ficou confortável em todos os aparelhos.
- Não prova o funcionamento por toque em Android e iOS reais.
- Não transforma o formulário em um agendamento persistido.
- Não comprova autenticação, autorização ou presença online.

Roteiro de demonstração do fluxo

1. Abrir Login e tocar em Entrar com Discord; explicar replace.
2. Na Home, mostrar saudação, categorias e total de partidas.
3. Selecionar uma categoria; repetir o toque e mostrar que o filtro sai.
4. Abrir uma partida; explicar seu serverId e a busca em Detalhes.
5. Voltar; conferir que não foi criada uma nova Home desnecessariamente.
6. Tocar em +; mostrar o formulário vazio e categorias apagadas.
7. Escolher duas categorias sucessivamente; mostrar a exclusividade.
8. Abrir o seletor, escolher Valorosos, reabrir e trocar o grupo.
9. Preencher dia, mês, hora, minuto e descrição; mostrar o teclado e a rolagem.
10. Fechar a lista sem escolher e confirmar que os campos permanecem.
11. Tocar em Agendar; ler a mensagem de demonstração.

12. Voltar à Home, abrir saída pelo avatar, cancelar e depois confirmar.

Esse roteiro ainda deve ser executado no aparelho que será usado na apresentação. Registre plataforma, tamanho de tela e resultado. Se uma ação não funcionar, relate o caso concreto; não substitua o teste por uma afirmação baseada apenas na leitura do código.

26. Roteiro para explicar o projeto ao professor

Abertura: propósito e limite

“O GrupoGame é uma demonstração de organização de partidas. Desenvolvemos quatro telas principais usando React Native com Expo e TypeScript. O objetivo foi reproduzir as referências e demonstrar componentes, navegação e estado local. Os dados são simulados e não existe autenticação real ou salvamento.”

Arquitetura: explique o motivo, não só o nome

“src/app define as rotas e coordena as telas. src/components contém partes repetidas ou grandes o suficiente para separar. src/data organiza os registros de demonstração e seus tipos. assets guarda imagens e fontes, enquanto referencias guarda o material visual. Mantive os estilos próximos da interface porque isso facilita ler o projeto nesta escala.”

Demonstração guiada pelo código

Abra _layout.tsx para explicar fontes e Stack. Em index.tsx, mostre JSX e o onPress. Em home.tsx, localize selectedCategoryId e visibleAppointments. Abra CategoryCard para mostrar como uma prop calculada altera o estilo. Em Detalhes, mostre useLocalSearchParams e find.

Em Agendar, siga a sequência completa: useState do ID, comparação em selected, callback que muda o estado e opacidade condicional. Depois mostre um TextInput controlado. Abra o modal de grupos e explique que onSelect retorna um ID à tela. Termine com fecharTeclado e a diferença entre ajustar espaço, rolar e dispensar o teclado.

Modelo de resposta para qualquer trecho

1. **Responsabilidade:** “Este trecho controla a escolha da categoria.”
2. **Entrada:** “Ele recebe o ID da opção tocada.”
3. **Transformação:** “O setter troca o ID guardado.”
4. **Resultado:** “A próxima renderização destaca apenas esse cartão.”
5. **Motivo:** “Um único ID evita seleções simultâneas.”

Como reagir a uma pergunta que você não sabe

Localize o componente, as props e a função chamada. Explique o que é verificável no código antes de supor uma resposta. Se for um recurso não implementado, diga isso: “Não há persistência nesta etapa; o estado dura enquanto a tela está montada.” É melhor delimitar o trabalho do que prometer um comportamento inexistente.

Não é necessário decorar cada tamanho e cada cor. É importante saber por que uma linha usa flexDirection, por que um campo é string, por que uma lista tem key e quem é o dono do estado. Use o

apêndice como consulta, não como texto para ler inteiro durante a apresentação.

27. Perguntas que podem aparecer na banca

Fundamentos

Qual a diferença entre React e React Native? React fornece componentes e o modelo de atualização da interface; React Native fornece componentes e integração para construir interfaces nas plataformas suportadas. Expo organiza ferramentas e módulos usados pelo projeto.

O que é JSX? É a sintaxe usada para descrever a árvore de componentes. As chaves permitem expressões JavaScript, e as tags recebem props.

Por que usar TypeScript? Para verificar contratos de dados e componentes durante o desenvolvimento. Ele ajuda a detectar usos incorretos, mas não valida uma data de calendário por si só.

O que diferencia prop de estado? A prop é recebida de fora do componente; o estado é mantido por uma instância. No CategoryCard, selected é prop. Na tela Agendar, selectedCategoryId é estado.

Arquitetura e componentes

Por que componentes não ficam em app? Porque essa pasta define rotas no Expo Router. Separá-los evita destinos acidentais e deixa claro o que é tela.

Por que os tipos ficam em data? Eles são pequenos e descrevem diretamente os registros simulados. Uma pasta types separada não resolveria uma necessidade atual.

O projeto usa MVC? Não há uma implementação formal desse padrão. A separação escolhida é por responsabilidade: rotas/telas, componentes e dados locais.

Por que não usou Redux? Os estados pertencem a telas específicas e o compartilhamento necessário é resolvido por props e callbacks. Não há estado global complexo que justifique a dependência.

Por que criar um modal separado se ele é usado só uma vez? Porque ele contém uma lista ou confirmação com árvore visual e estilos próprios, e a separação deixa a tela significativamente mais legível.

Estado e listas

Como garante apenas uma categoria selecionada? Guardando um único ID e comparando cada cartão a ele. Só um ID pode corresponder ao valor atual.

Por que a categoria da Home pode ser desmarcada e a de Agendar não? A Home alterna um filtro; Agendar escolhe uma opção do formulário. Cada tela passa um callback adequado à sua regra.

Qual a diferença entre map, filter e find? map transforma todos os itens; filter retorna um array de correspondências; find retorna a primeira correspondência ou undefined.

Por que usar key? Para identificar itens de forma estável entre renderizações. Um ID estável é mais apropriado que a posição em listas que mudam ou são filtradas.

Por que extraData? Para a FlatList considerar a mudança de um valor que afeta seus itens, como seleção, mesmo quando data mantém a mesma referência.

Os dados locais são estado? Não. São arrays exportados. O estado guarda escolhas e entradas do usuário; a tela consulta os arrays para obter informações.

Navegação e formulário

Por que passar somente um ID na rota? Para manter o endereço simples e buscar os dados em uma origem definida. O destino também consegue lidar com acesso direto e ID inexistente.

Qual a diferença entre back e replace? back usa o histórico para retornar; replace troca o destino atual. O fallback sem histórico usa replace para Home.

O modal cria outra tela na pilha? Não. Ele é uma sobreposição controlada por estado e mantém a tela anterior montada.

Por que o dia é uma string? Porque o campo pode estar vazio e precisa preservar zeros à esquerda. A entrada textual pode ser convertida e validada em uma etapa futura.

O teclado numérico valida o horário? Não. Ele facilita digitar. O código remove não dígitos e limita o comprimento, mas não valida intervalos de hora e minuto.

O que faz KeyboardAvoidingView? Ajusta o layout em resposta ao teclado. ScrollView permite alcançar o conteúdo e Keyboard.dismiss fecha o teclado. São funções diferentes.

Fechar o modal apaga o formulário? Não, porque os estados continuam em Agendar. Ao desmontar e reabrir a rota, os valores iniciais são recriados.

Limites e qualidade

Login e saída são reais? São fluxos visuais simulados. Não existe token, OAuth, proteção de rota ou sessão Discord.

Agendar salva na Home? Não. O botão apenas informa a demonstração. Não altera o array de partidas nem grava em banco.

Compilar significa que tudo foi testado? Não. TypeScript verifica tipos e a exportação gera bundles. Toque, teclado, aparência e acessibilidade exigem testes de interface.

Por que algumas capas são diferentes? Foram reutilizados assets locais disponíveis e registrados. A estrutura segue as referências, mas alguns recortes exatos não estavam disponíveis.

28. Exercícios e respostas esperadas

Use estes exercícios para treinar explicações e prever o comportamento. Eles não pedem alteração do aplicativo entregue.

1. Preveja a seleção

Agendar começa com null. O usuário toca em Duelo 1x1 e depois em Treino. Qual estado fica guardado e quais cartões ficam apagados?

Resposta: 'training'. Apenas Treino recebe selected=true. As outras categorias recebem o estilo de opacidade 0,4. Não existem três operações extras para desmarcá-las.

2. Compare as duas telas

Na Home, toque duas vezes em Diversão. Repita em Agendar. O que muda?

Resposta: a Home termina sem filtro, pois a segunda seleção retorna a null. Agendar continua com 'fun', porque o callback apenas atribui o ID tocado.

3. Explique a busca sem resultado

O endereço contém /servidor/inexistente. O que acontece?

Resposta: find não encontra um servidor. O retorno antecipado exibe a mensagem apropriada com voltar; o código não tenta renderizar players de undefined.

4. Diferencie callback de execução imediata

Por que onPress={handleBack} e onPress={() => handleSelectServer(item.id)} são válidos?

Resposta: ambos passam funções para executar no toque. O primeiro já possui tudo de que precisa; o segundo cria uma função para fornecer o ID do item. Chamar handleSelectServer(item.id) diretamente no valor da prop executaria a ação durante a renderização.

5. Pense no campo controlado

O usuário cola "a2b" no dia. O que a função de mudança tenta armazenar?

Resposta: ela remove os caracteres não numéricos do texto recebido. O maxLength também participa do processamento do input, então a entrada recebida pode ser limitada pela plataforma. O conceito a explicar é que a função armazena somente os dígitos que chegam ao callback, sem garantir uma data válida.

6. Separe aparências de regras

Se opacity fosse 0,4 mas onPress continuasse funcionando, o cartão estaria desabilitado?

Resposta: não. Estaria apenas apagado visualmente. disabled seria outra prop com efeito no comportamento de interação.

7. Demonstre o ciclo completo do servidor

Explique, sem ler o código, como o toque em Valorosos chega ao formulário.

Resposta: a linha do modal chama onSelect com o ID. Agendar recebe o callback, atualiza selectedServerId e fecha o modal. Na próxima renderização, find localiza Valorosos e o bloco mostra nome, imagem e jogo.

8. Identifique uma evolução e seu impacto

Se futuramente fosse necessário salvar partidas, bastaria mudar o texto da mensagem?

Resposta: não. Seriam necessários requisitos de validação, persistência, tratamento de falhas e atualização da fonte de dados da Home. Essa evolução não está implementada e exigiria novo escopo.

29. Glossário e mapa de consulta

Termo	Significado neste projeto
Componente	Função que descreve uma parte da interface
JSX	Sintaxe de composição visual com expressões JavaScript
Prop	Entrada recebida por um componente
Estado	Informação mantida entre renderizações da instância
Setter	Função usada para solicitar atualização de estado
Callback	Função fornecida para ser chamada em resposta a uma ação
Hook	Recurso como useState ou useEffect usado por componentes
Renderização	Cálculo da descrição visual a partir dos valores atuais
Dado derivado	Resultado calculado de dados, props ou estado
Rota	Caminho associado a uma tela
Stack	Organização de destinos em pilha
Parâmetro	Valor usado para identificar ou configurar um destino
Modal	Sobreposição temporária controlada pela interface
Asset	Arquivo de imagem, ícone, fonte ou outro recurso
Bundle	Código e recursos preparados para uma plataforma
Persistência	Armazenamento que sobrevive ao estado temporário da tela
Mock	Dado ou comportamento simulado para demonstração
Tipagem estática	Verificação de contratos de código antes da execução

Em qual arquivo encontro cada assunto?

Fontes e useEffect: `_layout.tsx`. Entrada simulada: `index.tsx`. Filtro e total: `home.tsx`. Parâmetro e servidor inexistente: `servidor/[id].tsx`. Formulário e teclado: `agendar.tsx`. Props e estilo condicional: `category-card.tsx`. ID enviado pelo card: `appointment-card.tsx`. Props opcionais: `screen-header.tsx`. Avatar alternativo: `player-item.tsx`. Lista de grupos: `server-select-modal.tsx`. Confirmação: `sign-out-modal.tsx`. Tipos e registros: os dois arquivos de data.

Como usar o apêndice

Os capítulos A01 a A13 contêm os arquivos funcionais completos e comentados. A14 a A16 contêm `package.json`, `tsconfig.json` e `app.json`. Não incluem `node_modules` nem os exemplos preservados do `template`, porque estes não compõem as quatro telas do trabalho. O material principal explica o papel desses arquivos preservados.

O código é uma fotografia desta edição. Se o projeto mudar depois, gere novamente a apostila para não apresentar uma versão antiga. A origem e o hash dos arquivos transcritos são registrados no arquivo de fontes gerado junto do PDF.

30. Referências e critérios de evidência

Fonte principal: o próprio projeto

Esta apostila foi construída a partir dos arquivos de `src/app`, `src/components` e `src/data`, das configurações da raiz, dos documentos de docs e das imagens de referencias. O código completo transcrito no apêndice é a principal evidência de como a implementação funciona. As decisões históricas foram conferidas com a implementação atual para não manter Valorosos como fixo ou o seletor como excluído.

Documentação técnica

- [Expo SDK 57](#): referência correspondente ao SDK declarado.
- [Expo Router](#): navegação: destinos, pilha e parâmetros.
- [Expo Font](#): carregamento de fontes locais.
- [Expo Symbols](#): nomes de símbolos por plataforma.
- [React Native 0.86: FlatList](#): listas, chaves e `extraData`.
- [React Native 0.86: Modal](#): sobreposição e fechamento.
- [React Native 0.86: TextInput](#): entradas de texto e props.
- [React Native 0.86: KeyboardAvoidingView](#): ajuste de layout para teclado.
- [React Native 0.86: ScrollView](#): conteúdo rolável e interação com teclado.
- [React: estado como fotografia](#): valores de cada renderização.
- [React: compartilhar estado](#): dono do estado e comunicação por props.
- [TypeScript: tipos do cotidiano](#): unions, arrays e tipos de objetos.

As explicações dos capítulos relacionam esses conceitos ao código do projeto; não são transcrições dos manuais. O conteúdo deve ser lido junto da versão instalada, pois APIs de Expo e React Native mudam entre versões.

Origem visual

Os protótipos foram fornecidos na pasta `referencias`. `assets/README.md` registra as fontes dos recursos locais, incluindo materiais educacionais `GamePlay/NLW` da Rocketseat, fontes Google Fonts e capas/logos usados na demonstração. Esta apostila não atribui a autoria dessas artes ao desenvolvimento do aplicativo.

Declaração de limites

Os registros de compilação e exportação são evidências técnicas das etapas executadas. Não foram tratados como prova de interação em aparelho ou comparação pixel a pixel. O app não possui autenticação real, banco de dados, persistência de formulário, calendário validado ou sincronização com servidores externos. Esses limites fazem parte de uma explicação correta do trabalho.

A01. Inicialização, fontes e Stack

Arquivo: src/app/_layout.tsx

Transcrição integral do arquivo local nesta edição. A coluna da esquerda numera as linhas originais; continuações visuais não recebem outro número. Não copie a numeração para o editor.

```
1  import { useFonts } from 'expo-font';
2  import { Stack } from 'expo-router';
3  import * as SplashScreen from 'expo-splash-screen';
4  import { StatusBar } from 'expo-status-bar';
5  import { useEffect } from 'react';
6
7  // Layout raiz: prepara as fontes e a navegação comum a todas as telas.
8  // Mantém a abertura do Expo visível enquanto as fontes locais são carregadas.
9  SplashScreen.preventAutoHideAsync();
10
11 export default function RootLayout() {
12   // Os nomes à esquerda são os mesmos usados em fontFamily nos StyleSheets.
13   const [fontsLoaded, fontError] = useFonts({
14     RajdhaniBold: require('@assets/fonts/Rajdhani-Bold.ttf'),
15     Inter: require('@assets/fonts/Inter.ttf'),
16   });
17
18   // useEffect executa este efeito quando fontsLoaded ou fontError muda.
19   // Em caso de erro, também liberamos a abertura para não bloquear o aplicativo.
20   useEffect(() => {
21     if (fontsLoaded || fontError) {
22       SplashScreen.hide();
23     }
24   }, [fontsLoaded, fontError]);
25
26   // Enquanto aguarda, não monta as telas com fontes ainda indisponíveis.
27   if (!fontsLoaded && !fontError) {
28     return null;
29   }
30
31   return (
32     <>
33     { /* Fragment agrupa elementos sem acrescentar uma View ao layout. */}
34     <StatusBar style="light" />
35     { /* Stack organiza rotas em pilha. Os arquivos de src/app definem as rotas;
36        Agendar e servidor/[id] também são descobertos automaticamente.
37        headerShown oculta o cabeçalho nativo porque usamos nosso próprio. */}
38     <Stack
39       screenOptions={{
40         headerShown: false,
41         contentStyle: { backgroundColor: '#0D133D' },
42       }}>
43       <Stack.Screen name="index" />
44       <Stack.Screen name="home" />
45     </Stack>
46   </>
47   );
48 }
```

A02. Tela de Login

Arquivo: src/app/index.tsx

Transcrição integral do arquivo local nesta edição. A coluna da esquerda numera as linhas originais; continuações visuais não recebem outro número. Não copie a numeração para o editor.

```

1  import { Image } from 'expo-image';
2  import { router } from 'expo-router';
3  import { Pressable, ScrollView, StyleSheet, Text, View } from 'react-native';
4  import { SafeAreaView } from 'react-native-safe-area-context';
5
6  // index.tsx corresponde à rota inicial (/). Um componente retorna a interface em JSX.
7  // Esta tela não precisa de useState: não possui campos nem dados que mudem localmente.
8  export default function LoginScreen() {
9    function handleSignIn() {
10     // Entrada simulada: não autentica nem acessa a conta do Discord.
11     // replace troca a rota atual: voltar da Home não retorna a este Login pela pilha.
12     router.replace('/home');
13   }
14
15   return (
16     <SafeAreaView style={styles.screen}>
17       {/* SafeAreaView respeita recortes/barras do aparelho; ScrollView permite
18        rolar em telas baixas. contentContainerStyle estiliza o conteúdo interno. */}
19       <ScrollView
20         contentContainerStyle={styles.scrollContent}
21         showsVerticalScrollIndicator={false}>
22         <View style={styles.container}>
23           {/* require inclui o arquivo local no bundle. contain preserva a imagem inteira.
24            accessible=false evita anunciar uma ilustração apenas decorativa. */}
25           <Image
26             source={require('@assets/images/login/illustration.png')}
27             style={styles.illustration}
28             contentFit="contain"
29             accessible={false}
30           />
31
32           <View style={styles.content}>
33             <Text style={styles.title} accessibilityRole="header">
34               Conecte-se{'\n'}e organize suas{'\n'}jogatinas
35             </Text>
36
37             <Text style={styles.description}>
38               Crie grupos para jogar seus games{'\n'}favoritos com seus amigos
39             </Text>
40
41             {/* Passamos a função a onPress, sem executá-la durante a renderização.
42              pressed é fornecido pelo Pressable para o feedback visual do toque.
43              As props de acessibilidade descrevem a ação ao leitor de tela. */}
44             <Pressable
45               accessibilityRole="button"
46               accessibilityLabel="Entrar com Discord"
47               accessibilityHint="Abre a Home em uma demonstração sem autenticação"
48               onPress={handleSignIn}
49               style={({ pressed }) => [styles.button, pressed && styles.buttonPressed]}>
50               <View style={styles.buttonIcon}>
51                 <Image
52                   source={require('@assets/images/login/discord.png')}
53                   style={styles.discordIcon}
54                   contentFit="contain"
55                   accessible={false}

```

```
56         />
57     </View>
58     <Text style={styles.buttonLabel}>Entrar com Discord</Text>
59 </Pressable>
60 </View>
61 </View>
62 </ScrollView>
63 </SafeAreaView>
64 );
65 }
66
67 // StyleSheet organiza os estilos. Números representam unidades lógicas de layout.
68 // flex: 1 ocupa o espaço disponível; flexGrow permite ao conteúdo crescer e rolar.
69 const styles = StyleSheet.create({
70   screen: {
71     flex: 1,
72     backgroundColor: '#0D133D',
73   },
74   scrollContent: {
75     flexGrow: 1,
76     justifyContent: 'center',
77     alignItems: 'center',
78     paddingTop: 24,
79     paddingBottom: 56,
80   },
81   container: {
82     width: '100%',
83     maxWidth: 420,
84   },
85   illustration: {
86     width: '100%',
87     aspectRatio: 375 / 360,
88   },
89   content: {
90     // Aproxima o título da ilustração, como na referência.
91     marginTop: -60,
92     paddingHorizontal: 24,
93     alignItems: 'center',
94   },
95   title: {
96     fontFamily: 'RajdhaniBold',
97     fontSize: 40,
98     lineHeight: 40,
99     color: '#DDE3F0',
100    textAlign: 'center',
101  },
102  description: {
103    marginTop: 16,
104    fontFamily: 'Inter',
105    fontSize: 16,
106    lineHeight: 25,
107    color: '#DDE3F0',
108    textAlign: 'center',
109  },
110  button: {
111    // row coloca ícone e texto lado a lado; alignItems centraliza na vertical.
112    alignSelf: 'stretch',
113    flexDirection: 'row',
114    alignItems: 'center',
115    minHeight: 56,
116    marginTop: 48,
117    marginHorizontal: 24,
118    backgroundColor: '#E91446',
119    borderRadius: 4,
120  },
121 }
```

```
121   buttonPressed: {
122     opacity: 0.75,
123   },
124   buttonIcon: {
125     width: 56,
126     alignSelf: 'stretch',
127     alignItems: 'center',
128     justifyContent: 'center',
129     borderRadius: 1,
130     borderRadiusColor: '#C3133D',
131   },
132   discordIcon: {
133     width: 24,
134     height: 18,
135   },
136   buttonLabel: {
137     // Ocupa o espaço restante depois do ícone, mantendo o texto centralizado.
138     flex: 1,
139     paddingHorizontal: 8,
140     paddingVertical: 16,
141     fontFamily: 'Inter',
142     fontSize: 14,
143     lineHeight: 20,
144     color: 'FFFFFF',
145     textAlign: 'center',
146   },
147 });
```

A03. Tela Home

Arquivo: src/app/home.tsx

Transcrição integral do arquivo local nesta edição. A coluna da esquerda numera as linhas originais; continuações visuais não recebem outro número. Não copie a numeração para o editor.

```

1  import { Image } from 'expo-image';
2  import { router } from 'expo-router';
3  import { useState } from 'react';
4  import { FlatList, Pressable, StyleSheet, Text, View } from 'react-native';
5  import { SafeAreaView } from 'react-native-safe-area-context';
6
7  import { AppointmentCard } from '@components/appointment-card';
8  import { CategoryCard } from '@components/category-card';
9  import { SignOutModal } from '@components/sign-out-modal';
10 import { appointments, categories, user, type CategoryId } from '@data/home';
11
12 export default function HomeScreen() {
13   // useState guarda valores entre renderizações. O setter solicita a atualização da tela.
14   // Um único ID (ou null) representa o filtro; o booleano controla somente a saída.
15   const [selectedCategoryId, setSelectedCategoryId] = useState<CategoryId | null>(null);
16   const [showSignOut, setShowSignOut] = useState(false);
17
18   // Valor derivado: calculamos a lista a partir do filtro, sem outro estado duplicado.
19   // filter cria um novo array; não remove partidas dos dados locais originais.
20   const visibleAppointments = selectedCategoryId
21     ? appointments.filter((appointment) => appointment.categoryId === selectedCategoryId)
22     : appointments;
23
24   function handleSelectCategory(categoryId: CategoryId) {
25     // Tocar novamente na mesma categoria remove o filtro.
26     // A forma funcional recebe o estado anterior para calcular o próximo.
27     setSelectedCategoryId((current) => current === categoryId ? null : categoryId);
28   }
29
30   function handleOpenServer(serverId: string) {
31     // O card entrega só o ID. [id] é o trecho variável da URL; Detalhes busca os dados.
32     // navigate abre/reutiliza o destino na pilha, evitando empilhar o mesmo destino ativo.
33     router.navigate({ pathname: '/servidor/[id]', params: { id: serverId } });
34   }
35
36   function handleSignOut() {
37     setShowSignOut(false);
38     // Saída simulada: substitui a Home pelo Login, sem sessão real do Discord.
39     router.replace('/');
40   }
41
42   return (
43     <SafeAreaView style={styles.screen}>
44       {/* FlatList renderiza a lista por itens. data fornece os dados, renderItem monta
45         cada card e keyExtractor identifica os itens com chaves estáveis.
46         O cabeçalho acompanha a rolagem porque pertence à própria lista. */}
47       <FlatList
48         style={styles.list}
49         contentContainerStyle={styles.listContent}
50         data={visibleAppointments}
51         keyExtractor={(item) => item.id}
52         showsVerticalScrollIndicator={false}
53         ListHeaderComponent={
54           <View>
55             <View style={styles.header}>

```

```

56      <Pressable
57        onPress={() => setShowSignOut(true)}
58        accessibilityRole="button"
59        accessibilityLabel="Sair"
60        accessibilityHint="Abre a confirmação de saída"
61        style={({ pressed }) => pressed && styles.avatarPressed}>
62      <Image source={user.avatar} style={styles.avatar} contentFit="cover" accessible={false}
  />
63    </Pressable>
64    <View style={styles.greeting}>
65      <Text style={styles.greetingText}>
66        Olá, <Text style={styles.userName}>{user.name}</Text>
67      </Text>
68      <Text style={styles.message}>{user.message}</Text>
69    </View>
70
71    <Pressable
72      onPress={() => router.navigate('/agendar')}
73      accessibilityRole="button"
74      accessibilityLabel="Agendar partida"
75      style={styles.addButton}>
76      <Text style={styles.addIcon}>+</Text>
77    </Pressable>
78  </View>
79
80  {/* Lista horizontal independente da vertical. extraData informa que a
81     seleção mudou mesmo que o array categories continue sendo o mesmo.
82     Props levam dados ao card; o callback onPress devolve o toque à tela. */}
83  <FlatList
84    horizontal
85    data={categories}
86    extraData={selectedCategoryId}
87    keyExtractor={({item}) => item.id}
88    showsHorizontalScrollIndicator={false}
89    contentContainerStyle={styles.categories}
90    renderItem={({ item }) => (
91      <CategoryCard
92        category={item}
93        selected={selectedCategoryId === item.id}
94        onPress={() => handleSelectCategory(item.id)}
95      />
96    )}
97  />
98
99  <View style={styles.sectionHeader}>
100    <Text style={styles.sectionTitle} accessibilityRole="header">Partidas agendadas</Text>
101    <Text style={styles.total}>Total {visibleAppointments.length}</Text>
102  </View>
103 </View>
104 }
105 renderItem={({ item }) => (
106   <AppointmentCard
107     appointment={item}
108     categoryLabel={categories.find((category) => category.id === item.categoryId)?.matchLabel ??
''}
109     onPress={handleOpenServer}
110   />
111 )}
112 ListEmptyComponent={<Text style={styles.empty}>Nenhuma partida nesta categoria.</Text>}
113 />
114 {/* O modal recebe estado e callbacks da Home; cancelar preserva o filtro. */}
115 <SignOutModal
116   visible={showSignOut}
117   onCancel={() => setShowSignOut(false)}
118   onConfirm={handleSignOut}

```

```
119     />
120   </SafeAreaView>
121 );
122 }
123
124 // Largura máxima mantém a leitura em telas grandes; padding dá espaço interno.
125 // Nos grupos em row, justifyContent distribui na horizontal e alignItems na vertical.
126 const styles = StyleSheet.create({
127   screen: {
128     flex: 1,
129     backgroundColor: '#0D133D',
130   },
131   list: {
132     width: '100%',
133     maxWidth: 600,
134     alignSelf: 'center',
135   },
136   listContent: {
137     paddingTop: 24,
138     paddingBottom: 16,
139   },
140   header: {
141     flexDirection: 'row',
142     alignItems: 'center',
143     paddingHorizontal: 24,
144     marginBottom: 40,
145     gap: 20,
146   },
147   avatar: {
148     width: 48,
149     height: 48,
150     borderRadius: 8,
151     borderWidth: 1,
152     borderColor: '#E91446',
153     backgroundColor: '#E91446',
154   },
155   avatarPressed: { opacity: 0.75 },
156   // A saudação ocupa o espaço restante entre o avatar e o botão +.
157   greeting: { flex: 1, gap: 4 },
158   greetingText: {
159     fontFamily: 'Inter',
160     fontSize: 20,
161     lineHeight: 28,
162     color: '#DDE3F0',
163   },
164   userName: { fontFamily: 'RajdhaniBold', fontSize: 24 },
165   message: {
166     fontFamily: 'Inter',
167     fontSize: 13,
168     lineHeight: 18,
169     color: '#ABB1CC',
170   },
171   addButton: {
172     width: 48,
173     height: 48,
174     alignItems: 'center',
175     justifyContent: 'center',
176     borderRadius: 8,
177     backgroundColor: '#E91446',
178   },
179   addIcon: {
180     fontFamily: 'Inter',
181     fontSize: 28,
182     lineHeight: 32,
183     color: '#FFFFFF',
```

```
184   },
185   categories: { paddingHorizontal: 24, gap: 8 },
186   sectionHeader: {
187     // wrap permite quebrar a linha se título e total não couberem lado a lado.
188     flexDirection: 'row',
189     flexWrap: 'wrap',
190     alignItems: 'center',
191     justifyContent: 'space-between',
192     gap: 8,
193     paddingHorizontal: 24,
194     marginTop: 40,
195     marginBottom: 24,
196   },
197   sectionTitle: {
198     fontFamily: 'RajdhaniBold',
199     fontSize: 20,
200     lineHeight: 26,
201     color: '#DDE3F0',
202   },
203   total: {
204     fontFamily: 'Inter',
205     fontSize: 13,
206     lineHeight: 20,
207     color: '#ABB1CC',
208   },
209   empty: {
210     paddingHorizontal: 24,
211     fontFamily: 'Inter',
212     fontSize: 14,
213     lineHeight: 22,
214     color: '#ABB1CC',
215   },
216 });
```

A04. Detalhes do servidor

Arquivo: src/app/servidor/[id].tsx

Transcrição integral do arquivo local nesta edição. A coluna da esquerda numera as linhas originais; continuações visuais não recebem outro número. Não copie a numeração para o editor.

```

1  import { Image } from 'expo-image';
2  import { router, useLocalSearchParams } from 'expo-router';
3  import { useState } from 'react';
4  import { FlatList, Pressable, Share, StyleSheet, Text, View } from 'react-native';
5  import { SafeAreaView } from 'react-native-safe-area-context';
6
7  import { PlayerItem } from '@components/player-item';
8  import { ScreenHeader } from '@components/screen-header';
9  import { servers } from '@data/servers';
10
11 // Gera os endereços dos servidores também na exportação estática para web.
12 export function generateStaticParams() {
13   return servers.map((server) => ({ id: server.id }));
14 }
15
16 export default function ServerDetailsScreen() {
17   // A rota recebe somente o ID. find devolve o registro correspondente ou undefined.
18   // O tipo admite parâmetro ausente/repetido; sem correspondência exibimos o aviso abaixo.
19   const { id } = useLocalSearchParams<{ id?: string | string[] }>();
20   const server = servers.find((item) => item.id === id);
21   // Ligamos a mensagem ao ID para não exibir feedback de outro servidor ao mudar a rota.
22   const [feedback, setFeedback] = useState<{ serverId: string; message: string } | null>(null);
23
24   function handleBack() {
25     // back retira a tela atual da pilha. Sem histórico (link direto), replace abre Home.
26     if (router.canGoBack()) router.back();
27     else router.replace('/home');
28   }
29
30   function handleJoin() {
31     // Guarda apenas uma mensagem de demonstração: não entra em uma partida real.
32     if (!server) return;
33     setFeedback({ serverId: server.id, message: 'Entrada simulada. Nenhuma conexão com o Discord foi realizada.' });
34   }
35
36   async function handleShare() {
37     // async/await aguarda a API de compartilhamento do sistema. Não cria convite Discord.
38     if (!server) return;
39     try {
40       await Share.share({ message: `${server.name}\n${server.description}` });
41     } catch (error) {
42       // Cancelamento não é tratado como falha; outros erros geram um aviso local.
43       if (error instanceof Error && error.name === 'AbortError') return;
44       setFeedback({ serverId: server.id, message: 'Compartilhamento indisponível neste dispositivo.' });
45     }
46   }
47
48   // Retorno antecipado evita acessar propriedades de um servidor inexistente.
49   if (!server) {
50     return (
51       <SafeAreaView style={styles.screen}>
52         <ScreenHeader title="Detalhes" onBack={handleBack} />
53         <Text style={styles.notFound}>Servidor não encontrado. Volte para escolher uma partida.</Text>
54       </SafeAreaView>

```

```

55     });
56   }
57
58   return (
59     <SafeAreaView style={styles.screen}>
60       <View style={styles.container}>
61         <ScreenHeader title="Detalhes" onBack={handleBack} onShare={handleShare} />
62         { /* O banner é o cabeçalho da lista; jogadores são itens com IDs próprios.
63            PlayerItem recebe cada jogador por prop, sem conhecer a navegação. */ }
64         <FlatList
65           style={styles.list}
66           data={server.players}
67           keyExtractor={({player}) => player.id}
68           renderItem={({ item }) => <PlayerItem player={item} />}
69           showsVerticalScrollIndicator={false}
70           ListHeaderComponent={
71             <View>
72               <View style={styles.banner}>
73                 <Image source={server.banner} style={StyleSheet.absoluteFill}
74                   contentFit="cover" accessible={false} />
75                 <View style={styles.bannerContent}>
76                   <Text style={styles.serverName} accessibilityRole="header">{server.name}</Text>
77                   <Text style={styles.description}>{server.description}</Text>
78                 </View>
79               </View>
80               <View style={styles.sectionHeader}>
81                 <Text style={styles.sectionTitle} accessibilityRole="header">Jogadores</Text>
82                 <Text style={styles.total}>Total {server.players.length}</Text>
83               </View>
84             </View>
85           }
86           ListEmptyComponent={<Text style={styles.notFound}>Nenhum jogador neste servidor.</Text>}
87         />
88         <View style={styles.footer}>
89           { /* && renderiza a mensagem somente quando pertence ao servidor atual.
90              ?. permite verificar feedback mesmo quando seu valor é null. */ }
91           {feedback?.serverId === server.id && (
92             <Text style={styles.feedback} accessibilityLiveRegion="polite">{feedback.message}</Text>
93           )}
94           <Pressable onPress={handleJoin} accessibilityRole="button"
95             accessibilityLabel="Entrar na partida"
96             accessibilityHint="Simula a entrada, sem conexão com o Discord"
97             style={({ pressed }) => [styles.button, pressed && styles.pressed]}>
98             <View style={styles.buttonIcon}>
99               <Image source={require('@assets/images/login/discord.png')}
100                 style={styles.discordIcon} contentFit="contain" accessible={false} />
101             </View>
102             <Text style={styles.buttonLabel}>Entrar na partida</Text>
103           </Pressable>
104         </View>
105       </View>
106     </SafeAreaView>
107   );
108 }
109
110 // A lista usa flex: 1 e ocupa o espaço entre cabeçalho e rodapé.
111 // absoluteFill cobre o banner com a imagem; o fundo translúcido ajuda a ler os textos.
112 const styles = StyleSheet.create({
113   screen: { flex: 1, backgroundColor: '#0D133D' },
114   container: { flex: 1, width: '100%', maxWidth: 600, alignSelf: 'center' },
115   list: { flex: 1 },
116   banner: { minHeight: 234, justifyContent: 'flex-end' },
117   bannerContent: { paddingHorizontal: 24, paddingVertical: 20, gap: 8, backgroundColor:
118     'rgba(13,19,61,0.45)' },
119   serverName: { fontFamily: 'RajdhaniBold', fontSize: 28, lineHeight: 34, color: '#DDE3F0' },

```

```
119   description: { fontFamily: 'Inter', fontSize: 13, lineHeight: 20, color: '#DDE3F0' },
120   sectionHeader: {
121     flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between',
122     flexWrap: 'wrap', gap: 8, paddingHorizontal: 24, marginTop: 24, marginBottom: 8,
123   },
124   sectionTitle: { fontFamily: 'RajdhaniBold', fontSize: 20, lineHeight: 26, color: '#DDE3F0' },
125   total: { fontFamily: 'Inter', fontSize: 12, lineHeight: 18, color: '#ABB1CC' },
126   footer: { paddingHorizontal: 24, paddingTop: 16, paddingBottom: 24, gap: 12 },
127   button: { flexDirection: 'row', alignItems: 'center', minHeight: 56, backgroundColor: '#E91446',
borderRadius: 8 },
128   buttonIcon: {
129     width: 56, alignSelf: 'stretch', alignItems: 'center', justifyContent: 'center',
130     borderRightWidth: 1, borderRightColor: '#C3133D',
131   },
132   discordIcon: { width: 24, height: 18 },
133   buttonLabel: {
134     flex: 1, paddingHorizontal: 8, paddingVertical: 16, textAlign: 'center',
135     fontFamily: 'Inter', fontSize: 14, lineHeight: 20, color: 'FFFFFF',
136   },
137   pressed: { opacity: 0.75 },
138   feedback: { fontFamily: 'Inter', fontSize: 12, lineHeight: 18, color: '#DDE3F0' },
139   notFound: { padding: 24, fontFamily: 'Inter', fontSize: 14, lineHeight: 22, color: '#DDE3F0' },
140 });
```

A05. Formulário Agendar

Arquivo: src/app/agendar.tsx

Transcrição integral do arquivo local nesta edição. A coluna da esquerda numera as linhas originais; continuações visuais não recebem outro número. Não copie a numeração para o editor.

```
1  import { Image } from 'expo-image';
2  import { router } from 'expo-router';
3  import { SymbolView } from 'expo-symbols';
4  import { useState } from 'react';
5  import {
6    Keyboard,
7    KeyboardAvoidingView,
8    Platform,
9    Pressable,
10   ScrollView,
11   StyleSheet,
12   Text,
13   TextInput,
14   TouchableWithoutFeedback,
15   View,
16 } from 'react-native';
17 import { SafeAreaView } from 'react-native-safe-area-context';
18
19 import { CategoryCard } from '@components/category-card';
20 import { ScreenHeader } from '@components/screen-header';
21 import { ServerSelectModal } from '@components/server-select-modal';
22 import { categories, type CategoryId } from '@data/home';
23 import { servers } from '@data/servers';
24
25 export default function ScheduleScreen() {
26   // Um único ID garante seleção de apenas uma categoria; null é o estado inicial vazio.
27   // useState mantém o valor entre renderizações e seu setter atualiza a interface.
28   const [selectedCategoryId, setSelectedCategoryId] = useState<CategoryId | null>(null);
29   // A escolha do servidor e a abertura da lista são estados independentes.
30   const [selectedServerId, setSelectedServerId] = useState<string | null>(null);
31   const [showServerSelect, setShowServerSelect] = useState(false);
32   // Derivamos o objeto pelo ID, sem duplicar nome, jogo e imagem em outros estados.
33   const selectedServer = servers.find((server) => server.id === selectedServerId);
34   // TextInput trabalha com strings: permitem vazio e zeros à esquerda, como "06".
35   const [day, setDay] = useState('');
36   const [month, setMonth] = useState('');
37   const [hour, setHour] = useState('');
38   const [minute, setMinute] = useState('');
39   const [description, setDescription] = useState('');
40   // Controla apenas a mensagem de demonstração do botão Agendar.
41   const [showFeedback, setShowFeedback] = useState(false);
42
43   function handleBack() {
44     // Retira a tela da pilha; se veio por link direto sem histórico, substitui por Home.
45     if (router.canGoBack()) router.back();
46     else router.replace('/home');
47   }
48
49   function fecharTeclado() {
50     // Remove o foco/fecha o teclado, mas não limpa os estados dos campos.
51     Keyboard.dismiss();
52   }
53
54   function handleSchedule() {
55     fecharTeclado();
```

```

56 // Apenas feedback local: não salva nem adiciona uma partida à Home.
57 setShowFeedback(true);
58 }
59
60 function handleOpenServers() {
61 // A lista aparece sobre o formulário; os campos continuam montados e preservados.
62 fecharTeclado();
63 setShowServerSelect(true);
64 }
65
66 function handleSelectServer(serverId: string) {
67 // Callback recebido pelo modal: guarda o ID escolhido e fecha a lista.
68 setSelectedServerId(serverId);
69 setShowServerSelect(false);
70 }
71
72 // padding ajusta o espaço inferior quando o teclado sobrepõe a tela nativa.
73 // A ScrollView permite alcançar o conteúdo quando a área disponível diminui.
74 // Na web, o navegador controla o viewport; o KeyboardAvoidingView fica desativado.
75 return (
76 <KeyboardAvoidingView
77 style={styles.screen}
78 enabled={Platform.OS !== 'web'}
79 behavior="padding">
80 <SafeAreaView style={styles.safeArea}>
81 <View style={styles.container}>
82 <ScreenHeader title="Agendar partida" onBack={handleBack} />
83
84 {/* handled permite tocar nos botões com teclado aberto; arrastar a rolagem
85 também pode dispensá-lo. O comportamento interactive é usado no iOS. */}
86 <ScrollView
87 style={styles.scroll}
88 contentContainerStyle={styles.scrollContent}
89 bounces={false}
90 showsVerticalScrollIndicator={false}
91 keyboardShouldPersistTaps="handled"
92 keyboardDismissMode={Platform.OS === 'ios' ? 'interactive' : 'on-drag'}>
93 {/* Como no exemplo: tocar no espaço livre fecha o teclado. */}
94 <TouchableWithoutFeedback onPress={fecharTeclado} accessible={false}>
95 <View style={styles.formContent}>
96 <Text style={[[styles.label, styles.categoryLabel]]}
97 accessibilityRole="header">Categoria</Text>
98 <ScrollView
99 horizontal
100 style={styles.categoryScroll}
101 showsHorizontalScrollIndicator={false}
102 keyboardShouldPersistTaps="handled"
103 contentContainerStyle={styles.categories}>
104 {/* map transforma o array pequeno em cartões; key identifica cada um.
105 A tela calcula selected e passa um callback. Trocar o ID faz a
106 categoria anterior perder o destaque, sem quatro booleanos. */}
107 {categories.map((category) => (
108 <CategoryCard
109 key={category.id}
110 category={category}
111 selected={selectedCategoryId === category.id}
112 onPress={() => setSelectedCategoryId(category.id)}
113 showSelectionIndicator
114 dimUnselected
115 accessibilityHint="Seleciona esta categoria para a partida. Apenas uma pode ser
116 selecionada."
117 />
118 )
119 )}
120 </ScrollView>

```

```

119         <View style={styles.form}>
120             {/* Renderização condicional: sem escolha há um espaço para a imagem.
121              ?. acessa o nome se existir servidor; ?? fornece o texto inicial. */}
122             <Pressable onPress={handleOpenServers} accessibilityRole="button"
123                 accessibilityLabel={selectedServer ? `Servidor: ${selectedServer.name},
124                 ${selectedServer.game}` : 'Selecione um servidor'}
125                 accessibilityHint="Abre a lista para escolher ou trocar o servidor"
126                 style={{({ pressed }) => [styles.server, pressed && styles.pressed]}}
127                 {selectedServer ? (
128                     <Image source={selectedServer.image} style={styles.serverImage} contentFit="cover"
129                     accessible={false} />
130                     ) : <View style={styles.serverImage} />}
131                 <View style={styles.serverInfo}>
132                     <Text style={styles.serverName}>{selectedServer?.name ?? 'Selecione um
133                     servidor'}</Text>
134                     {selectedServer && <Text style={styles.serverGame}>{selectedServer.game}</Text>}
135                 </View>
136                 <SymbolView name={{ ios: 'chevron.right', android: 'chevron_right', web:
137                 'chevron_right' }}
138                     size={18} tintColor="#ABB1CC" style={styles.serverChevron} />
139             </Pressable>
140
141             <View style={styles.dateAndTime}>
142                 <View style={styles.fieldGroup}>
143                     <Text style={styles.label}>Dia e mês</Text>
144                     <View style={styles.inputRow}>
145                         {/* Campo controlado: value lê o estado e onChangeText o atualiza.
146                          A expressão regular remove tudo que não é dígito. O teclado
147                          numérico facilita digitar; maxLength limita a dois caracteres.
148                          Isso não valida se a data ou o horário realmente existem. */}
149                         <TextInput
150                             accessibilityLabel="Dia" value={day}
151                             onChangeText={({text}) => setDay(text.replace(/\D/g, ''))}
152                             keyboardType="number-pad" maxLength={2}
153                             selectionColor="#E91446" style={styles.numberInput}
154                         />
155                         <Text style={styles.separator}></Text>
156                         <TextInput
157                             accessibilityLabel="Mês" value={month}
158                             onChangeText={({text}) => setMonth(text.replace(/\D/g, ''))}
159                             keyboardType="number-pad" maxLength={2}
160                             selectionColor="#E91446" style={styles.numberInput}
161                         />
162                     </View>
163                 </View>
164
165                 <View style={styles.fieldGroup}>
166                     <Text style={styles.label}>Horário</Text>
167                     <View style={styles.inputRow}>
168                         <TextInput
169                             accessibilityLabel="Hora" value={hour}
170                             onChangeText={({text}) => setHour(text.replace(/\D/g, ''))}
171                             keyboardType="number-pad" maxLength={2}
172                             selectionColor="#E91446" style={styles.numberInput}
173                         />
174                         <Text style={styles.separator}></Text>
175                         <TextInput
176                             accessibilityLabel="Minuto" value={minute}
177                             onChangeText={({text}) => setMinute(text.replace(/\D/g, ''))}
178                             keyboardType="number-pad" maxLength={2}
179                             selectionColor="#E91446" style={styles.numberInput}
180                         />
181                     </View>
182                 </View>
183             </View>

```

```

180
181     <View style={styles.descriptionHeader}>
182       <Text style={styles.label}>Descrição</Text>
183       <Text style={styles.limit}>Max 100 caracteres</Text>
184     </View>
185     {/* multiline permite várias linhas; maxLength limita a 100 caracteres.
186       setDescription pode ser passado diretamente porque recebe o texto. */}
187     <TextInput
188       accessibilityLabel="Descrição da partida"
189       accessibilityHint="Máximo de 100 caracteres"
190       value={description}
191       onChangeText={setDescription}
192       multiline
193       maxLength={100}
194       selectionColor="#E91446"
195       style={styles.descriptionInput}
196     />
197   </View>
198
199   <View style={styles.footer}>
200     {/* && só inclui a mensagem no JSX depois do toque em Agendar. */}
201     {showFeedback && (
202       <Text style={styles.feedback} accessibilityLiveRegion="polite">
203         Demonstração: nenhum agendamento foi salvo.
204       </Text>
205     )}
206     <Pressable
207       onPress={handleSchedule}
208       accessibilityRole="button"
209       accessibilityLabel="Agendar"
210       accessibilityHint="Simulação local, sem salvar um agendamento"
211       style={({ pressed }) => [styles.button, pressed && styles.pressed]}>
212       <Text style={styles.buttonLabel}>Agendar</Text>
213     </Pressable>
214   </View>
215 </View>
216 </TouchableWithoutFeedback>
217 </ScrollView>
218 </View>
219 </SafeAreaView>
220 {/* O modal recebe props, sem criar outra rota. Fechar não redefine os campos. */}
221 <ServerSelectModal visible={showServerSelect} selectedServerId={selectedServerId}
222   onSelect={handleSelectServer} onClose={() => setShowServerSelect(false)} />
223 </KeyboardAvoidingView>
224 );
225 }
226
227 // StyleSheet centraliza a aparência; os valores de estado decidem o conteúdo exibido.
228 const styles = StyleSheet.create({
229   screen: { flex: 1, backgroundColor: '#0D133D' },
230   safeArea: { flex: 1 },
231   container: { flex: 1, width: '100%', maxWidth: 600, alignSelf: 'center' },
232   scroll: { flex: 1 },
233   // Preenche a altura livre, mas deixa o conteúdo crescer para permitir rolagem.
234   scrollContent: { flexGrow: 1, paddingTop: 24, paddingBottom: 24 },
235   formContent: { flexGrow: 1 },
236   label: { fontFamily: 'RajdhaniBold', fontSize: 18, lineHeight: 24, color: '#DDE3F0' },
237   categoryLabel: { marginHorizontal: 24, marginBottom: 12 },
238   // A faixa horizontal não deve esticar nem encolher na direção vertical.
239   categoryScroll: { flexGrow: 0, flexShrink: 0 },
240   categories: { paddingHorizontal: 24, gap: 8 },
241   form: { paddingHorizontal: 24, marginTop: 28 },
242   server: {
243     // row alinha imagem, texto e seta. overflow recorta a imagem nos cantos arredondados.
244     flexDirection: 'row', alignItems: 'center', minHeight: 68,

```

```
245     borderWidth: 1, borderColor: '#243189', borderRadius: 8, overflow: 'hidden',
246   },
247   serverImage: { width: 64, height: 68, backgroundColor: '#1B245F' },
248   serverInfo: { flex: 1, paddingHorizontal: 16, paddingVertical: 8, gap: 4 },
249   serverName: { fontFamily: 'RajdhaniBold', fontSize: 20, lineHeight: 24, color: '#DDE3F0' },
250   serverGame: { fontFamily: 'Inter', fontSize: 13, lineHeight: 20, color: '#ABB1CC' },
251   serverChevron: { marginRight: 16 },
252   dateAndTime: {
253     // Os grupos ficam lado a lado; wrap permite quebrar em telas estreitas.
254     flexDirection: 'row', flexWrap: 'wrap', justifyContent: 'space-between',
255     gap: 20, marginTop: 28,
256   },
257   fieldGroup: { gap: 12 },
258   inputRow: { flexDirection: 'row', alignItems: 'center', gap: 4 },
259   numberInput: {
260     width: 48, minHeight: 48, padding: 8, textAlign: 'center',
261     fontFamily: 'Inter', fontSize: 14, color: '#DDE3F0',
262     borderRadius: 8, borderWidth: 1, borderColor: '#243189', backgroundColor: '#1B245F',
263   },
264   separator: { fontFamily: 'Inter', fontSize: 16, color: '#ABB1CC' },
265   descriptionHeader: {
266     flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between',
267     flexWrap: 'wrap', gap: 8, marginTop: 24, marginBottom: 12,
268   },
269   limit: { fontFamily: 'Inter', fontSize: 12, lineHeight: 18, color: '#ABB1CC' },
270   descriptionInput: {
271     minHeight: 120, padding: 12, textAlignVertical: 'top',
272     fontFamily: 'Inter', fontSize: 14, lineHeight: 22, color: '#DDE3F0',
273     borderRadius: 8, borderWidth: 1, borderColor: '#243189', backgroundColor: '#1B245F',
274   },
275   // Empurra o botão para baixo quando há espaço; ele continua dentro da rolagem.
276   footer: { marginTop: 'auto', paddingTop: 40, paddingHorizontal: 24, gap: 12 },
277   button: { minHeight: 56, justifyContent: 'center', alignItems: 'center', borderRadius: 8,
    backgroundColor: '#E91446' },
278   buttonLabel: { padding: 16, fontFamily: 'Inter', fontSize: 14, lineHeight: 20, color: 'FFFFFF' },
279   pressed: { opacity: 0.75 },
280   feedback: { fontFamily: 'Inter', fontSize: 12, lineHeight: 18, color: '#DDE3F0' },
281 });
```

A06. Cartão de categoria

Arquivo: src/components/category-card.tsx

Transcrição integral do arquivo local nesta edição. A coluna da esquerda numera as linhas originais; continuações visuais não recebem outro número. Não copie a numeração para o editor.

```

1  import { Image } from 'expo-image';
2  import { Pressable, StyleSheet, Text, View } from 'react-native';
3
4  import type { Category } from '@data/home';
5
6  // Props são entradas recebidas da tela. O tipo define os dados e callbacks permitidos.
7  // ? indica prop opcional; as opções visuais têm padrões que preservam a Home.
8  type CategoryCardProps = {
9    category: Category;
10   selected: boolean;
11   onPress: () => void;
12   accessibilityHint?: string;
13   showSelectionIndicator?: boolean;
14   dimUnselected?: boolean;
15 };
16
17 // Componente controlado pela tela: não guarda seleção própria em useState.
18 // Home usa o toque para filtrar; Agendar usa o mesmo cartão para escolher uma categoria.
19 export function CategoryCard({
20   category,
21   selected,
22   onPress,
23   accessibilityHint = 'Filtra as partidas. Toque novamente para mostrar todas.',
24   showSelectionIndicator = false,
25   dimUnselected = false,
26 }: CategoryCardProps) {
27   return (
28     <Pressable
29       onPress={onPress}
30       accessibilityRole="button"
31       accessibilityLabel={category.title}
32       accessibilityState={{ selected }}
33       accessibilityHint={accessibilityHint}
34       // Estilos são combinados na ordem do array; os posteriores sobreescrevem os anteriores.
35       // && aplica cada estilo só quando sua condição é verdadeira.
36       style={({ pressed }) => [
37         styles.card,
38         selected && styles.selected,
39         showSelectionIndicator && selected && styles.selectedWithIndicator,
40         pressed && styles.pressed,
41         // Por último para que o toque não clareie uma categoria ainda não selecionada.
42         dimUnselected && !selected && styles.unselected,
43       ]>
44       {showSelectionIndicator && (
45         <View style={[styles.indicator, selected && styles.indicatorSelected]} />
46       )}
47       <Image source={category.icon} style={styles.icon} contentFit="contain" accessible={false} />
48       <Text style={styles.title}>{category.title}</Text>
49     </Pressable>
50   );
51 }
52
53 // O layout padrão da View é em coluna: ícone e título ficam um abaixo do outro.
54 const styles = StyleSheet.create({
55   card: {

```

```
56     width: 104,
57     minHeight: 120,
58     paddingHorizontal: 8,
59     paddingVertical: 18,
60     alignItems: 'center',
61     justifyContent: 'space-between',
62     gap: 16,
63     borderRadius: 8,
64     borderWidth: 1,
65     borderColor: '#243189',
66     backgroundColor: '#1B245F',
67   },
68   selected: {
69     borderColor: '#E91446',
70     backgroundColor: '#243189',
71   },
72   selectedWithIndicator: {
73     borderColor: '#243189',
74   },
75   indicator: {
76     // Posicionamento absoluto prende o marcador ao canto sem deslocar ícone/texto.
77     position: 'absolute',
78     top: 8,
79     right: 8,
80     width: 7,
81     height: 7,
82     borderRadius: 2,
83     borderWidth: 1,
84     borderColor: '#243189',
85   },
86   indicatorSelected: {
87     backgroundColor: '#E91446',
88     borderColor: '#E91446',
89   },
90   pressed: {
91     opacity: 0.75,
92   },
93   unselected: {
94     // Apaga o cartão inteiro visualmente; não desabilita seu onPress.
95     opacity: 0.4,
96   },
97   icon: {
98     width: 48,
99     height: 48,
100  },
101  title: {
102    fontFamily: 'RajdhaniBold',
103    fontSize: 15,
104    lineHeight: 20,
105    color: '#DDE3F0',
106    textAlign: 'center',
107  },
108  });
```

A07. Cartão de partida

Arquivo: src/components/appointment-card.tsx

Transcrição integral do arquivo local nesta edição. A coluna da esquerda numera as linhas originais; continuações visuais não recebem outro número. Não copie a numeração para o editor.

```

1  import { Image } from 'expo-image';
2  import { Pressable, StyleSheet, Text, View } from 'react-native';
3
4  import type { Appointment } from '@data/home';
5
6  // O card recebe os dados da partida, o rótulo da categoria e uma ação opcional.
7  // A assinatura do callback exige um ID de servidor e não retorna um valor (void).
8  type AppointmentCardProps = {
9    appointment: Appointment;
10   categoryLabel: string;
11   onPress?: (serverId: string) => void;
12 };
13
14 export function AppointmentCard({ appointment, categoryLabel, onPress }: AppointmentCardProps) {
15   // Valores calculados a partir das props: não precisam de estados independentes.
16   const role = appointment.isHost ? 'Anfitrião' : 'Visitante';
17   const roleColor = appointment.isHost ? '#E91446' : '#32BD50';
18
19   return (
20     <Pressable
21       // Sem callback, o card fica desabilitado. ?. chama a função somente se existir.
22       // A função anônima adia a chamada e entrega o ID à Home quando houver um toque.
23       disabled={!onPress}
24       onPress={() => onPress?.(appointment.serverId)}
25       accessible
26       accessibilityRole="button"
27       accessibilityState={{ disabled: !onPress }}
28       accessibilityLabel={` ${appointment.title}, ${categoryLabel}, ${appointment.date} às
${appointment.time}, ${role}`}
29       style={({ pressed }) => [styles.card, pressed && styles.pressed]}>
30       <Image source={appointment.image} style={styles.cover} contentFit="cover" accessible={false} />
31
32       <View style={styles.details}>
33         <View style={styles.titleRow}>
34           <Text style={styles.title}>{appointment.title}</Text>
35           <Text style={styles.category}>{categoryLabel}</Text>
36         </View>
37
38         <View style={styles.metadata}>
39           <View style={styles.dateGroup}>
40             <Image
41               source={require('@assets/images/home/calendar.svg')}
42               style={styles.smallIcon}
43               contentFit="contain"
44               accessible={false}
45             />
46             <Text style={styles.date}>{appointment.date} às {appointment.time}</Text>
47           </View>
48
49           <View style={styles.roleGroup}>
50             <Image
51               source={require('@assets/images/home/player.svg')}
52               style={styles.smallIcon}
53               contentFit="contain"
54               tintColor={roleColor}

```

```
55         accessible={false}
56       />
57       <Text style={[[styles.role, { color: roleColor }]]}>{role}</Text>
58     </View>
59   </View>
60 </View>
61 </Pressable>
62 );
63 }
64
65 // Card em linha: capa com tamanho fixo e details com flex: 1 para o espaço restante.
66 // wrap acomoda metadados em outra linha; tintColor reaproveita o ícone com outra cor.
67 const styles = StyleSheet.create({
68   card: {
69     flexDirection: 'row',
70     alignItems: 'flex-start',
71     gap: 20,
72     paddingHorizontal: 24,
73     paddingBottom: 30,
74   },
75   pressed: { opacity: 0.75 },
76   cover: {
77     width: 64,
78     height: 68,
79     borderRadius: 8,
80     borderWidth: 1,
81     borderColor: '#243189',
82     backgroundColor: '#171F52',
83   },
84   details: {
85     flex: 1,
86     minHeight: 70,
87     paddingTop: 4,
88     paddingBottom: 12,
89     borderBottomWidth: 1,
90     borderBottomColor: '#1B2565',
91     gap: 10,
92   },
93   titleRow: {
94     flexDirection: 'row',
95     flexWrap: 'wrap',
96     alignItems: 'center',
97     justifyContent: 'space-between',
98     columnGap: 8,
99     rowGap: 4,
100  },
101  title: {
102    flexShrink: 1,
103    fontFamily: 'RajdhaniBold',
104    fontSize: 20,
105    lineHeight: 24,
106    color: '#DDE3F0',
107  },
108  category: {
109    fontFamily: 'Inter',
110    fontSize: 12,
111    lineHeight: 18,
112    color: '#ABB1CC',
113  },
114  metadata: {
115    flexDirection: 'row',
116    flexWrap: 'wrap',
117    justifyContent: 'space-between',
118    columnGap: 8,
119    rowGap: 6,
```

```
120   },
121   dateGroup: {
122     flexDirection: 'row',
123     alignItems: 'center',
124     gap: 6,
125   },
126   roleGroup: {
127     flexDirection: 'row',
128     alignItems: 'center',
129     gap: 6,
130   },
131   smallIcon: { width: 16, height: 16 },
132   date: {
133     fontFamily: 'Inter',
134     fontSize: 12,
135     lineHeight: 18,
136     color: '#DDE3F0',
137   },
138   role: {
139     fontFamily: 'Inter',
140     fontSize: 12,
141     lineHeight: 18,
142   },
143 });
```

A08. Cabeçalho

Arquivo: src/components/screen-header.tsx

Transcrição integral do arquivo local nesta edição. A coluna da esquerda numera as linhas originais; continuações visuais não recebem outro número. Não copie a numeração para o editor.

```

1  import { SymbolView } from 'expo-symbols';
2  import { Pressable, StyleSheet, Text, View } from 'react-native';
3
4  // Cabeçalho reutilizado em Detalhes e Agendar. A tela fornece o título e as ações.
5  // onShare é opcional: Agendar não passa essa função e não mostra compartilhar.
6  type Props = {
7    title: string;
8    onBack: () => void;
9    onShare?: () => void;
10 };
11
12 export function ScreenHeader({ title, onBack, onShare }: Props) {
13   // O componente não decide a rota: apenas chama os callbacks recebidos por props.
14   return (
15     <View style={styles.header}>
16       <Pressable onPress={onBack} accessibilityRole="button" accessibilityLabel="Voltar"
17         style={({ pressed }) => [styles.action, pressed && styles.pressed]}>
18         <SymbolView name={{ ios: 'arrow.left', android: 'arrow_back', web: 'arrow_back' }}
19           size={24} tintColor="#DDE3F0" />
20       </Pressable>
21       <Text style={styles.title} accessibilityRole="header">{title}</Text>
22       /* Sem compartilhar, a View vazia reserva a largura do botão para centralizar
23        o título. SymbolView recebe nomes diferentes para os ícones de cada plataforma. */
24       {onShare ? (
25         <Pressable onPress={onShare} accessibilityRole="button" accessibilityLabel="Compartilhar
servidor"
26           style={({ pressed }) => [styles.action, pressed && styles.pressed]}>
27           <SymbolView name={{ ios: 'square.and.arrow.up', android: 'share', web: 'share' }}
28             size={24} tintColor="#E91446" />
29         </Pressable>
30       ) : <View style={styles.action} />}
31     </View>
32   );
33 }
34
35 // Ações com 48 de largura nas pontas; o título ocupa o centro usando flex: 1.
36 const styles = StyleSheet.create({
37   header: {
38     minHeight: 64, paddingHorizontal: 12, flexDirection: 'row',
39     alignItems: 'center', backgroundColor: '#1B245F',
40   },
41   action: { width: 48, height: 48, alignItems: 'center', justifyContent: 'center' },
42   title: {
43     flex: 1, textAlign: 'center', fontFamily: 'RajdhaniBold',
44     fontSize: 20, lineHeight: 26, color: '#DDE3F0',
45   },
46   pressed: { opacity: 0.7 },
47 });

```

A09. Jogador

Arquivo: src/components/player-item.tsx

Transcrição integral do arquivo local nesta edição. A coluna da esquerda numera as linhas originais; continuações visuais não recebem outro número. Não copie a numeração para o editor.

```

1  import { Image } from 'expo-image';
2  import { StyleSheet, Text, View } from 'react-native';
3
4  import type { Player } from '@data/servers';
5
6  // Cada item recebe um jogador por prop; não consulta API nem altera presença real.
7  type Props = { player: Player };
8
9  export function PlayerItem({ player }: Props) {
10   // Um valor derivado controla o texto e a cor de status, sem useState extra.
11   const available = player.status === 'available';
12
13   return (
14     <View style={styles.row}>
15       /* O ternário escolhe foto quando disponível ou as iniciais como alternativa. */
16       {player.avatar ? (
17         <Image source={player.avatar} style={styles.avatar} contentFit="cover" accessible={false} />
18       ) : (
19         <View style={styles.avatar}>
20           <Text style={styles.initials} accessible={false}>{player.initials}</Text>
21         </View>
22       )}
23       <View style={styles.details}>
24         <Text style={styles.name}>{player.name}</Text>
25         <View style={styles.status}>
26           <View style={[styles.dot, { backgroundColor: available ? '#32BD50' : '#E91446' }]} />
27           <Text style={styles.statusText}>{available ? 'Disponível' : 'Ocupado'}</Text>
28         </View>
29       </View>
30     </View>
31   );
32 }
33
34 // row organiza avatar/textos em linha; details cresce para ocupar o espaço restante.
35 // A combinação de width/height iguais e borderRadius pela metade desenha o ponto.
36 // hairlineWidth fornece uma borda fina adequada à densidade da tela.
37 const styles = StyleSheet.create({
38   row: { flexDirection: 'row', alignItems: 'center', gap: 16, paddingLeft: 24 },
39   avatar: {
40     width: 48, height: 48, borderRadius: 8, backgroundColor: '#24336B',
41     borderWidth: 1, borderColor: '#3C4980', alignItems: 'center', justifyContent: 'center',
42   },
43   initials: { fontFamily: 'RajdhaniBold', fontSize: 20, color: '#DDE3F0' },
44   details: {
45     flex: 1, gap: 4, paddingVertical: 16, paddingRight: 24,
46     borderBottomWidth: StyleSheet.hairlineWidth, borderBottomColor: '#24305B',
47   },
48   name: { fontFamily: 'RajdhaniBold', fontSize: 18, lineHeight: 22, color: '#DDE3F0' },
49   status: { flexDirection: 'row', alignItems: 'center', gap: 8 },
50   dot: { width: 8, height: 8, borderRadius: 4 },
51   statusText: { fontFamily: 'Inter', fontSize: 12, lineHeight: 18, color: '#ABB1CC' },
52 });

```

A10. Seleção de servidor

Arquivo: src/components/server-select-modal.tsx

Transcrição integral do arquivo local nesta edição. A coluna da esquerda numera as linhas originais; continuações visuais não recebem outro número. Não copie a numeração para o editor.

```

1  import { Image } from 'expo-image';
2  import { SymbolView } from 'expo-symbols';
3  import { FlatList, Modal, Pressable, StyleSheet, Text, View } from 'react-native';
4  import { SafeAreaView } from 'react-native-safe-area-context';
5
6  import { servers } from '@data/servers';
7
8  // Agendar controla estes valores. O modal só apresenta dados e comunica ações.
9  // onSelect recebe apenas o ID; onClose fecha sem trocar o servidor selecionado.
10 type Props = {
11   visible: boolean;
12   selectedServerId: string | null;
13   onSelect: (serverId: string) => void;
14   onClose: () => void;
15 };
16
17 export function ServerSelectModal({ visible, selectedServerId, onSelect, onClose }: Props) {
18   // Modal é uma sobreposição, não uma rota. visible controla sua exibição.
19   // onRequestClose trata o voltar do Android enquanto o modal está aberto.
20   return (
21     <Modal visible={visible} transparent animationType="slide" statusBarTranslucent
22     onRequestClose={onClose}>
23       <View style={styles.overlay}>
24         /* O fundo ocupa a tela e permite cancelar. A área do traço também fecha,
25         oferecendo uma ação nomeada para leitores de tela. Não há gesto de arrastar. */
26         <Pressable style={StyleSheet.absoluteFill} onPress={onClose} accessible={false} />
27         <SafeAreaView edges={['bottom', 'left', 'right']} style={styles.sheet}>
28           <View style={styles.content} accessibilityViewIsModal>
29             <Pressable onPress={onClose} accessibilityRole="button" accessibilityLabel="Fechar seleção
30 de servidor"
31             style={styles.handleArea}>
32               <View style={styles.handle} />
33             </Pressable>
34             /* keyExtractor identifica linhas pelo ID. extraData atualiza a indicação
35             acessível de seleção quando só selectedServerId muda.
36             renderItem transforma cada registro em uma linha clicável. */
37             <FlatList
38               data={servers}
39               keyExtractor={({server}) => server.id}
40               extraData={selectedServerId}
41               accessibilityLabel="Selecione um servidor"
42               showsVerticalScrollIndicator={false}
43               contentContainerStyle={styles.list}
44               renderItem={({ item }) => (
45                 <Pressable
46                   // A função só é chamada ao tocar; Agendar recebe o ID e fecha o modal.
47                   onPress={() => onSelect(item.id)}
48                   accessibilityRole="button"
49                   accessibilityLabel={` ${item.name}, ${item.isAdmin ? 'Administrador' : 'Convidado'}`}
50                   accessibilityState={{ selected: item.id === selectedServerId }}
51                   accessibilityHint="Seleciona este servidor e retorna ao formulário"
52                   style={{ pressed }} => [styles.row, pressed && styles.pressed]}>
53                     <Image source={item.image} style={styles.image} contentFit="cover" accessible={false}
54
55                   />
56                 <View style={styles.info}>

```

```

53         <View style={styles.text}>
54             <Text style={styles.name}><{item.name}</Text>
55             <Text style={styles.role}><{item.isAdmin ? 'Administrador' : 'Convidado'}</Text>
56         </View>
57         <SymbolView name={{ ios: 'chevron.right', android: 'chevron_right', web:
'chevron_right' }}
58             size={18} tintColor="#ABB1CC" />
59         </View>
60     </Pressable>
61     }}
62     />
63 </View>
64 </SafeAreaView>
65 </View>
66 </Modal>
67 );
68 }
69
70 // Overlay cobre o formulário. flex-end posiciona o painel na base da tela.
71 // A altura de 88% deixa parte da tela anterior visível; FlatList rola o que não couber.
72 const styles = StyleSheet.create({
73     overlay: { flex: 1, justifyContent: 'flex-end', backgroundColor: 'rgba(0,0,0,0.75)' },
74     sheet: { height: '88%', width: '100%', maxWidth: 600, alignSelf: 'center', backgroundColor: '#0D133D'
},
75     content: { flex: 1 },
76     handleArea: { height: 112, alignItems: 'center', paddingTop: 12 },
77     handle: { width: 36, height: 2, borderRadius: 1, backgroundColor: '#526DFF' },
78     list: { paddingHorizontal: 24, paddingBottom: 24 },
79     row: { flexDirection: 'row', alignItems: 'center', gap: 20, minHeight: 92 },
80     image: { width: 64, height: 68, borderRadius: 8, backgroundColor: '#1B245F' },
81     // A borda pertence às informações, começando após a imagem como no protótipo.
82     info: { flex: 1, minHeight: 92, flexDirection: 'row', alignItems: 'center', gap: 8,
83         paddingVertical: 16, borderTopWidth: 1, borderTopColor: '#1B245F' },
84     text: { flex: 1, gap: 4 },
85     name: { fontFamily: 'RajdhaniBold', fontSize: 18, lineHeight: 24, color: '#DDE3F0' },
86     role: { fontFamily: 'Inter', fontSize: 12, lineHeight: 18, color: '#ABB1CC' },
87     pressed: { opacity: 0.7 },
88 });

```

A11. Confirmação de saída

Arquivo: src/components/sign-out-modal.tsx

Transcrição integral do arquivo local nesta edição. A coluna da esquerda numera as linhas originais; continuações visuais não recebem outro número. Não copie a numeração para o editor.

```

1  import { Modal, Pressable, StyleSheet, Text, View } from 'react-native';
2  import { SafeAreaView } from 'react-native-safe-area-context';
3
4  // visible vem do estado da Home. Os callbacks dizem o que fazer ao cancelar/confirmar.
5  // O modal não conhece o Router nem mantém uma segunda cópia desse estado.
6  type Props = {
7    visible: boolean;
8    onCancel: () => void;
9    onConfirm: () => void;
10 };
11
12 export function SignOutModal({ visible, onCancel, onConfirm }: Props) {
13   // transparent permite enxergar o fundo escurecido; fade anima a aparição.
14   // onRequestClose usa o mesmo cancelamento para o botão voltar do Android.
15   return (
16     <Modal
17       visible={visible}
18       transparent
19       animationType="fade"
20       statusBarTranslucent
21       onRequestClose={onCancel}>
22       <View style={styles.overlay}>
23         {/* absoluteFill cobre a área externa; tocar nela cancela sem sair da Home. */}
24         <Pressable style={StyleSheet.absoluteFill} onPress={onCancel} accessible={false} />
25         <SafeAreaView edges={['bottom', 'left', 'right']} style={styles.sheet}>
26           <View style={styles.content} accessibilityViewIsModal>
27             <Text style={styles.title} accessibilityRole="header">
28               Deseja sair do Game<Text style={styles.highlight}>Play</Text>?
29             </Text>
30             {/* onPress recebe a função: ela só roda após o toque. A Home decide
31              fechar o modal ou substituir a rota pelo Login. */}
32             <View style={styles.actions}>
33               <Pressable
34                 onPress={onCancel}
35                 accessibilityRole="button"
36                 accessibilityLabel="Não, permanecer na Home"
37                 style={({ pressed }) => [styles.button, styles.cancelButton, pressed && styles.pressed]}>
38                 <Text style={styles.buttonLabel}>Não</Text>
39               </Pressable>
40               <Pressable
41                 onPress={onConfirm}
42                 accessibilityRole="button"
43                 accessibilityLabel="Sim, sair e voltar ao Login"
44                 style={({ pressed }) => [styles.button, styles.confirmButton, pressed &&
45 styles.pressed]}>
46                 <Text style={styles.buttonLabel}>Sim</Text>
47               </Pressable>
48             </View>
49           </SafeAreaView>
50         </View>
51       </Modal>
52     );
53   }
54

```

```
55 // O painel fica na base por flex-end. actions em row e os dois botões com flex: 1
56 // dividem a largura igualmente; gap mantém o intervalo entre eles.
57 const styles = StyleSheet.create({
58   overlay: { flex: 1, justifyContent: 'flex-end', backgroundColor: 'rgba(0,0,0,0.75)' },
59   sheet: { width: '100%', maxWidth: 600, alignSelf: 'center', backgroundColor: '#0D133D' },
60   content: { paddingHorizontal: 24, paddingTop: 24, paddingBottom: 24, gap: 24 },
61   title: { fontFamily: 'RajdhaniBold', fontSize: 20, lineHeight: 26, color: '#DDE3F0', textAlign:
'center' },
62   highlight: { color: '#E91446' },
63   actions: { flexDirection: 'row', gap: 8 },
64   button: { flex: 1, minHeight: 56, alignItems: 'center', justifyContent: 'center', borderRadius: 8 },
65   cancelButton: { borderWidth: 1, borderColor: '#243189' },
66   confirmButton: { backgroundColor: '#E91446' },
67   buttonLabel: { fontFamily: 'Inter', fontSize: 14, lineHeight: 20, color: '#DDE3F0', padding: 12 },
68   pressed: { opacity: 0.75 },
69 });
```

A12. Dados de Home e categorias

Arquivo: src/data/home.ts

Transcrição integral do arquivo local nesta edição. A coluna da esquerda numera as linhas originais; continuações visuais não recebem outro número. Não copie a numeração para o editor.

```

1  import type { ImageSource } from 'expo-image';
2
3  // União de strings: TypeScript aceita somente estes quatro IDs de categoria.
4  // Tipos ajudam a encontrar erros durante o desenvolvimento; não são dados de uma API.
5  export type CategoryId = 'ranked' | 'duel' | 'fun' | 'training';
6
7  export type Category = {
8    id: CategoryId;
9    title: string;
10   matchLabel: string;
11   // Aceita uma fonte de imagem do Expo ou o identificador de um asset local via require.
12   icon: ImageSource | number;
13 };
14
15 // id identifica a partida na lista; serverId liga a partida aos dados de servers.ts.
16 // categoryId permite filtrar e localizar o rótulo da categoria sem duplicá-lo.
17 export type Appointment = {
18   id: string;
19   serverId: string;
20   title: string;
21   categoryId: CategoryId;
22   date: string;
23   time: string;
24   isHost: boolean;
25   image: ImageSource | number;
26 };
27
28 // Perfil fictício usado na saudação. Não existe sessão autenticada neste projeto.
29 export const user = {
30   name: 'Tiago',
31   message: 'Hoje é dia de vitória',
32   avatar: require('@assets/images/home/avatar.png'),
33 };
34
35 // Array compartilhado entre Home e Agendar. [] no tipo significa lista de Category.
36 // export permite importar os mesmos dados em vários componentes.
37 export const categories: Category[] = [
38   { id: 'ranked', title: 'Ranqueada', matchLabel: 'Ranqueada', icon:
39     require('@assets/images/home/ranked.svg') },
40   { id: 'duel', title: 'Duelo 1x1', matchLabel: '1x1', icon: require('@assets/images/home/duel.svg') },
41   { id: 'fun', title: 'Diversão', matchLabel: 'Diversão', icon: require('@assets/images/home/fun.svg') },
42   { id: 'training', title: 'Treino', matchLabel: 'Treino', icon:
43     require('@assets/images/home/training.svg') },
44 ];
45
46 // Dados de demonstração: não são eventos reais nem dependem de uma API.
47 export const appointments: Appointment[] = [
48   {
49     id: 'match-1', serverId: 'lendarios', title: 'Lendários', categoryId: 'ranked',
50     date: '18/06', time: '21:00', isHost: true,
51     image: require('@assets/images/home/league-of-legends.png'),
52   },
53   {
54     id: 'match-2', serverId: 'yeah-boy', title: 'Yeah, boy', categoryId: 'fun',

```

```
53     date: '23/06', time: '19:00', isHost: false,
54     image: require('@assets/images/home/red-dead-redemption-2.jpg'),
55   },
56   {
57     id: 'match-3', serverId: 'rumo-ao-topo', title: 'Rumo ao topo', categoryId: 'duel',
58     date: '20/06', time: '09:00', isHost: true,
59     image: require('@assets/images/home/counter-strike.png'),
60   },
61   {
62     id: 'match-4', serverId: 'bora-queimar-tudo', title: 'Bora queimar tudo', categoryId: 'ranked',
63     date: '20/06', time: '14:20', isHost: true,
64     image: require('@assets/images/home/apex-legends.png'),
65   },
66   {
67     id: 'match-5', serverId: 'valorosos', title: 'Valorosos', categoryId: 'fun',
68     date: '18/06', time: '21:00', isHost: true,
69     image: require('@assets/images/home/valorant.png'),
70   },
71   {
72     // A sexta partida não está visível na referência; este item é fictício.
73     id: 'match-6', serverId: 'exploradores', title: 'Exploradores', categoryId: 'training',
74     date: '25/06', time: '20:00', isHost: false,
75     image: require('@assets/images/home/world-of-warcraft.png'),
76   },
77 ];
```

A13. Servidores e jogadores

Arquivo: src/data/servers.ts

Transcrição integral do arquivo local nesta edição. A coluna da esquerda numera as linhas originais; continuações visuais não recebem outro número. Não copie a numeração para o editor.

```
1 import type { ImageSource } from 'expo-image';
2
3 // Contrato dos dados de um jogador. avatar? é opcional: sem foto mostramos iniciais.
4 // status é uma união de valores permitidos, sem presença online real.
5 export type Player = {
6   id: string;
7   name: string;
8   initials: string;
9   avatar?: ImageSource | number;
10  status: 'available' | 'busy';
11 };
12
13 // Registro compartilhado por Detalhes e pelo seletor de Agendar.
14 // image é a miniatura; banner é a imagem grande de Detalhes; game é o nome do jogo.
15 // isAdmin indica papel fictício no grupo, diferente de isHost (anfitrião da partida).
16 export type Server = {
17   id: string;
18   name: string;
19   description: string;
20   game: string;
21   image: ImageSource | number;
22   isAdmin: boolean;
23   banner: ImageSource | number;
24   players: Player[];
25 };
26
27 // O mesmo grupo fictício participa dos servidores desta demonstração.
28 const players: Player[] = [
29   {
30     id: 'tiago', name: 'Tiago Luchtenberg', initials: 'TL', status: 'available',
31     avatar: require('@assets/images/home/avatar.png'),
32   },
33   { id: 'rodrigo', name: 'Rodrigo Gonçalves', initials: 'RG', status: 'busy' },
34   { id: 'diego', name: 'Diego Fernandes', initials: 'DF', status: 'busy' },
35 ];
36
37 // IDs únicos ligam a rota, a seleção em Agendar e os cards da Home.
38 // Estes registros são constantes locais: as telas consultam com find e não os salvam.
39 export const servers: Server[] = [
40   {
41     id: 'lendarios', name: 'Lendários',
42     game: 'League of Legends', image: require('@assets/images/home/league-of-legends.png'), isAdmin:
43     false,
44     description: 'É hoje que vamos chegar ao challenger sem perder uma partida de md10',
45     banner: require('@assets/images/servers/banner.png'), players,
46   },
47   {
48     id: 'yeah-boy', name: 'Yeah, boy',
49     game: 'Red Dead Redemption 2', image: require('@assets/images/home/red-dead-redemption-2.jpg'),
50     isAdmin: false,
51     description: 'Reúna os amigos para explorar o Velho Oeste e se divertir em equipe.',
52     banner: require('@assets/images/home/red-dead-redemption-2.jpg'), players,
53   },
54   {
55     id: 'rumo-ao-topo', name: 'Rumo ao topo',
```

```
54     game: 'Counter-Strike', image: require('@assets/images/home/counter-strike.png'), isAdmin: true,
55     description: 'Um duelo entre amigos para treinar a mira e melhorar a cada rodada.',
56     banner: require('@assets/images/home/counter-strike.png'), players,
57   },
58   {
59     id: 'bora-queimar-tudo', name: 'Bora queimar tudo',
60     game: 'Apex Legends', image: require('@assets/images/home/apex-legends.png'), isAdmin: false,
61     description: 'Prepare seu esquadrão para buscar a vitória na próxima partida.',
62     banner: require('@assets/images/home/apex-legends.png'), players,
63   },
64   {
65     id: 'valorosos', name: 'Valorosos',
66     game: 'Valorant', image: require('@assets/images/home/valorant.png'), isAdmin: false,
67     description: 'Uma partida entre amigos para praticar estratégias e jogar juntos.',
68     banner: require('@assets/images/home/valorant.png'), players,
69   },
70   {
71     id: 'exploradores', name: 'Exploradores',
72     game: 'World of Warcraft', image: require('@assets/images/home/world-of-warcraft.png'), isAdmin:
false,
73     description: 'Vamos explorar novos caminhos e treinar em grupo nesta aventura.',
74     banner: require('@assets/images/home/world-of-warcraft.png'), players,
75   },
76 ];
```

A14. Dependências e comandos

Arquivo: package.json

Transcrição integral do arquivo local nesta edição. A coluna da esquerda numera as linhas originais; continuações visuais não recebem outro número. Não copie a numeração para o editor.

```
1  {
2    "name": "grupogame-app",
3    "main": "expo-router/entry",
4    "version": "1.0.0",
5    "dependencies": {
6      "@expo/ui": "~57.0.19",
7      "expo": "~57.0.24",
8      "expo-constants": "~57.0.19",
9      "expo-device": "~57.0.2",
10     "expo-font": "~57.0.4",
11     "expo-glass-effect": "~57.0.3",
12     "expo-image": "~57.0.5",
13     "expo-linking": "~57.0.10",
14     "expo-router": "~57.0.22",
15     "expo-splash-screen": "~57.0.9",
16     "expo-status-bar": "~57.0.1",
17     "expo-symbols": "~57.0.3",
18     "expo-system-ui": "~57.0.4",
19     "expo-web-browser": "~57.0.3",
20     "react": "19.2.3",
21     "react-dom": "19.2.3",
22     "react-native": "0.86.3",
23     "react-native-gesture-handler": "~2.32.0",
24     "react-native-reanimated": "4.5.1",
25     "react-native-safe-area-context": "~5.7.0",
26     "react-native-screens": "~4.26.0",
27     "react-native-web": "~0.21.0",
28     "react-native-worklets": "0.10.1"
29   },
30   "devDependencies": {
31     "@types/react": "~19.2.2",
32     "typescript": "~6.0.3"
33   },
34   "scripts": {
35     "start": "expo start",
36     "reset-project": "node ./scripts/reset-project.js",
37     "android": "expo start --android",
38     "ios": "expo start --ios",
39     "web": "expo start --web",
40     "lint": "expo lint"
41   },
42   "private": true
43 }
```

A15. Configuração do TypeScript

Arquivo: tsconfig.json

Transcrição integral do arquivo local nesta edição. A coluna da esquerda numera as linhas originais; continuações visuais não recebem outro número. Não copie a numeração para o editor.

```
1  {
2    "extends": "expo/tsconfig.base",
3    "compilerOptions": {
4      "strict": true,
5      "paths": {
6        "@/*": [
7          "./src/*"
8        ],
9        "@/assets/*": [
10         "./assets/*"
11       ]
12     }
13   },
14   "include": [
15     "**/*.ts",
16     "**/*.tsx",
17     ".expo/types/**/*.ts",
18     "expo-env.d.ts"
19   ]
20 }
```

A16. Configuração do Expo

Arquivo: app.json

Transcrição integral do arquivo local nesta edição. A coluna da esquerda numera as linhas originais; continuações visuais não recebem outro número. Não copie a numeração para o editor.

```
1  {
2    "expo": {
3      "name": "grupogame-app",
4      "slug": "grupogame-app",
5      "version": "1.0.0",
6      "orientation": "portrait",
7      "icon": "./assets/images/icon.png",
8      "scheme": "grupogameapp",
9      "userInterfaceStyle": "automatic",
10     "ios": {
11       "icon": "./assets/expo.icon"
12     },
13     "android": {
14       "adaptiveIcon": {
15         "backgroundColor": "#E6F4FE",
16         "foregroundImage": "./assets/images/android-icon-foreground.png",
17         "backgroundImage": "./assets/images/android-icon-background.png",
18         "monochromeImage": "./assets/images/android-icon-monochrome.png"
19       },
20       "predictiveBackGestureEnabled": false
21     },
22     "web": {
23       "output": "static",
24       "favicon": "./assets/images/favicon.png"
25     },
26     "plugins": [
27       "expo-router",
28       [
29         "expo-splash-screen",
30         {
31           "backgroundColor": "#208AEF",
32           "image": "./assets/images/splash-icon.png",
33           "imageWidth": 76
34         }
35       ]
36     ],
37     "experiments": {
38       "typedRoutes": true,
39       "reactCompiler": true
40   }
41 }
42 }
```